Phân Tích Kỹ Thuật AI Agent

https://gemini.google.com/app/3c7525ae4e5dbcb1

User prompt: Phân tích nghiên cứu tìm hiểu xử lý nội dung https://youtu.be/EWSUvEyFwjc 1 người – 20+ AI agent – hơn 2.000 pull request mỗi tháng. Lauren Tan đã làm điều đó thế nào? Lauren Tan (biệt danh "poteto") từng làm engineering manager ở Netflix, làm React ở Meta, sau đó sang Cursor và hiện dẫn dắt Grok Bot tại SpaceXAI. Cô cũng là tác giả của pstack, một bộ công cụ mã nguồn mở giúp AI agent làm việc có quy trình như một kỹ sư thực thụ. Trong buổi livestream gần đây với Matt Pocock, cô kể lại hành trình từ chỗ ngồi canh từng dòng code của một agent đến chỗ điều hành cả một "đội" AI làm việc suốt ngày đêm. Lauren không thích ví việc này với một nhà máy sản xuất code. Cô chọn hình ảnh căn bếp của một nhà hàng Michelin. Một người nấu ăn ở nhà có thể tự đi chợ, sơ chế, nấu và dọn dẹp. Nhưng khi căn bếp có thêm nhiều đầu bếp, người ta cần dụng cụ, cần phân công, cần quy trình đáng tin cậy, và bếp trưởng phải chịu trách nhiệm cho mọi món ăn được mang ra bàn. Quản lý nhiều AI agent, theo cô, cũng đòi hỏi đúng những điều đó. Mọi thứ bắt đầu từ một nỗi bực mình rất đời thường. Khi mới vào Cursor và được giao xử lý hiệu năng, Lauren tự mở công cụ đo đạc, tự đọc số liệu rồi kể lại cho AI nghe. AI sửa code, cô chạy thử, cô quan sát, cô giải thích kết quả. Cô nhận ra mình đã thành một "người chuyển lời", và chừng nào mọi vòng lặp còn phải đi qua tay cô thì chẳng thể nào chạy nhiều việc song song. Bước ngoặt đến khi cô trao cho AI khả năng tự mở ứng dụng, tự thao tác như một người dùng, tự thu thập bằng chứng và tự đánh giá xem thay đổi của mình có thật sự hiệu quả hay không. Từ đó, "xong việc" không còn là vài dòng test màu xanh, mà là một kết quả có thể nhìn thấy tận mắt. Nhưng khi AI đã biết tự kiểm tra, một sự lãng phí khác lại lộ ra. Mỗi lần nhận việc, agent lại tự viết một đoạn script mới để điều khiển ứng dụng, dùng xong rồi bỏ, và agent sau lại làm lại theo một kiểu khác. Lauren gom những thao tác cố định ấy thành công cụ dùng chung. Phần việc máy móc giao cho công cụ, còn AI chỉ cần tập trung vào phần đòi hỏi phán đoán: kết hợp bằng chứng, chọn hướng đi, cân nhắc đánh đổi. Công cụ giúp AI làm việc tốt hơn, nhưng chính cấu trúc của codebase mới quyết định AI sẽ viết ra thứ gì. Lauren kể rằng phiên bản đầu tiên của Grok Bot chỉ gồm khoảng tám file khổng lồ, mỗi file dài hơn mười nghìn dòng. AI cứ thế nhét thêm code vào đó, vì đó là con đường dễ nhất. Cô đã xây lại toàn bộ theo một cấu trúc chặt chẽ: chia theo tính năng, quy ước rõ ràng, luật kiểm tra nghiêm ngặt. Triết lý của cô rất đơn giản: khi AI cứ lặp lại một lỗi, đừng nhắc nó mãi, hãy thay đổi môi trường để lỗi đó không còn chỗ xảy ra. Thời gian ngồi giám sát từng dòng code nên được dành để mài sắc công cụ. Khi một agent đã có thể hoàn thành công việc một cách đáng tin, câu hỏi tiếp theo là: việc ở đâu ra? Lauren chia quy trình của mình thành hai vòng. Vòng trong là nơi agent hiểu mục tiêu, sửa code và kiểm tra kết quả. Vòng ngoài là cả một thế giới bug report, yêu cầu mới và thông tin phát sinh đến từ Slack, Linear, X hay email. Để nối hai vòng này, cô dùng những agent đóng vai "chánh văn phòng", chuyên gom thông tin, giao việc, cung cấp bối cảnh và theo dõi tiến độ, giống như một bếp trưởng nắm rõ từng order và biết ai đang lo món nào. Nhờ vậy, cô chưa bao giờ phải tự tay mở 2.500 cuộc chat. Tuy vậy, nhanh không có nghĩa là vội. Có bug report nào đến là giao ngay cho agent sửa nghe thì hợp lý, nhưng nhiều báo cáo khác nhau có khi lại chung một nguyên nhân gốc. Sửa từng cái một chỉ sinh ra một đống bản vá trùng lặp và che mờ vấn đề thật. Lauren cho agent ghi lại các phát hiện vào một tài liệu, rồi cứ vài ngày cô xem lại một lần để tìm ra điểm chung, trước khi quyết định sửa tận gốc. Đến đây, câu hỏi ban đầu lại quay về: cô có xem hết chỗ PR đó không? Câu trả lời là cô kiểm tra theo cách lấy mẫu, sau khi code đã được merge. Cô soi kỹ những đường tắt, những mẹo vá tạm, những mẫu code xấu, và hễ thấy lỗi lặp lại thì cô sửa chính hệ thống đã sinh ra nó. Cô có hơn mười agent điều phối chạy 24/7, mỗi agent phụ trách một mảng. Đổi lại, việc dùng nhiều agent để kiểm chứng lẫn nhau rất tốn token. Và cô nói rất thẳng: cài pstack vào không có nghĩa là bạn sẽ có ngay kết quả như cô. Đó là thành quả của cả một quá trình dài tinh chỉnh. Cũng chính vì mọi thứ xoay quanh kiểm chứng, nên cách làm này có giới hạn của nó. Với những thay đổi có thể merge rồi hoàn tác, mọi chuyện khá dễ. Nhưng với những thay đổi có thể làm mất dữ liệu hay gây ra hậu quả khó đảo ngược thì sao? Lauren thừa nhận cô chưa có câu trả lời chung. Kết quả càng dễ kiểm chứng thì AI càng có thể được giao nhiều quyền; còn việc nào khó kiểm chứng thì mức tự chủ tương tự vẫn còn rất xa. Vậy người bình thường nên bắt đầu từ đâu? Lauren khuyên hãy mở lại những cuộc trò chuyện cũ với AI và tìm những lần mình phải sửa đi sửa lại, phải giải thích lại bối cảnh, phải tự tay làm thay. Chính những lần can thiệp ấy là nguyên liệu để tạo nên kỹ năng mới cho AI. Cô tin rằng khi mô hình ngày càng mạnh, những kỹ năng này sẽ ngắn dần đi. Nhưng mục tiêu, gu thẩm mỹ và tiêu chí thế nào là "đúng" thì vẫn phải do con người nói rõ, và càng phải rõ hơn khi AI ngày càng giỏi thực thi. Nhìn lại toàn bộ câu chuyện, con số 2.500 PR hóa ra không phải điều đáng chú ý nhất. Điều đáng chú ý là cách Lauren từng bước xây dựng niềm tin: tìm một việc mình hay phải làm tay, cho AI cách tự làm và tự kiểm chứng, đóng gói quy trình đó thành công cụ và quy tắc, rồi mới mở rộng dần. AI có thể nấu rất nhiều món, nhưng bếp trưởng vẫn là người chọn thực đơn và chịu trách nhiệm cho từng đĩa ăn được mang ra bàn. Còn bạn thì sao? Bạn đang ngồi canh từng dòng code khi dùng AI, hay đã dám giao hẳn việc cho nó? Nếu một ngày AI được tự merge code vào sản phẩm thật, bạn sẽ thấy yên tâm hay lo lắng? Hãy chia sẻ suy nghĩ của bạn ở phần bình luận nhé 👇 #AI #AIAgent #LaurenTan #pstack #LàmViệcVớiAI

Response: Bài viết bạn cung cấp là bản tóm tắt tổng hợp từ chuỗi chia sẻ của **Lauren Tan** (biệt danh `@poteto` — cựu Engineering Manager tại Netflix, cựu kỹ sư đội ngũ React Core tại Meta, từng làm việc tại Cursor và hiện dẫn dắt **Grok Bot** tại SpaceX / xAI).

Khi đối chiếu trực tiếp với video gốc Lauren Tan - SpaceXAI engineer và mã nguồn bộ công cụ **pstack**, có thể thấy bài viết tiếng Việt đã tóm lược tốt mặt **triết lý vận hành**, nhưng đã lược bỏ phần quan trọng nhất đối với dân kỹ thuật: **cấu trúc kỹ thuật bên dưới (under-the-hood engineering)** giúp cô ấy dám để AI tự động merge code thẳng vào nhánh `main` mà không làm sập hệ thống.

Dưới đây là bản phân tích, bóc tách chuyên sâu từng lớp kỹ thuật, quy trình và kiến trúc hệ thống từ video cùng hệ sinh thái `pstack`.

1. Đối chiếu nguồn dữ liệu & Bối cảnh thực tế

Trước khi đi vào chi tiết kỹ thuật, cần làm rõ mối liên hệ giữa **video đính kèm** và **bài viết tóm tắt**:

- **Bối cảnh trong video (http://www.youtube.com/watch?v=EWSUvEyFwjc):** Đây là buổi chia sẻ dài 55 phút của Lauren Tan ngay sau ngày ra mắt **Grok Bot** [32:21] và mô hình **Grok 4.6** [50:54]. Tại thời điểm quay video này (5 tháng sau khi gia nhập), cô vừa chạm mốc **1.000 PR vào tháng trước** và **gần 800 PR chỉ trong 12 ngày đầu tháng tiếp theo** [05:15], với khoảng **20 PR được AI tự động merge (auto-merge) mỗi đêm** khi cô đang ngủ [04:02].
- **Sự phát triển lên con số 2.000 – 2.500 PR/tháng:** Con số hơn 2.000 PR (chính xác là khoảng 2.500 PR/tháng) và khái niệm *"Chief of Staff agents"* (Agent Chánh văn phòng) là kết quả cuối tháng đó, được cô hệ thống hóa lại trong bài nói chuyện tại Cursor Compile và buổi livestream tiếp theo với Matt Pocock.

2. Giải phẫu 5 Trụ cột Kỹ thuật đứng sau hiệu suất 2.000+ PR/tháng

Trụ cột 1: "Đường cong Niềm tin" (The Trust Curve) — Từ Micromanager đến Bếp trưởng

Lauren chỉ ra nghịch lý phổ biến khi kỹ sư dùng AI: tiết kiệm được một chút thời gian gõ code nhưng lại tốn nhiều thời gian hơn để đọc, bắt lỗi và sửa lại những gì AI làm sai [00:54]. Khi AI liên tục "ảo giác" (hallucinate) và tự tin tuyên bố đã tìm ra nguyên nhân gốc (*smoking gun*) tới lần thứ 100 nhưng thực chất hoàn toàn đoán mò, kỹ sư sẽ mất niềm tin [01:54].

Cô liên hệ trực tiếp với kinh nghiệm làm Quản lý Kỹ thuật (Engineering Manager):

- **Trạng thái thiếu niềm tin (Micromanagement):** Khi bạn không tin tưởng nhân sự (hoặc AI agent), bạn buộc phải đứng sau lưng giám sát từng dòng lệnh [02:24]. Chừng nào con người còn nằm trong vòng lặp giám sát trực tiếp từng bước, bạn không thể chạy song song 10 hay 100 agent vì ngay cả đầu ra của 1 agent bạn còn chưa dám tin [03:30].
- **Mô hình Bếp trưởng (Head Chef):** Kỹ sư thời đại Agent không còn là người tự tay nấu từng món, mà là Bếp trưởng thiết kế khu bếp, phân chia các trạm (stations) cho bếp phó và phụ bếp [22:48]. Trách nhiệm của kỹ sư chuyển từ *viết code* sang *thiết kế môi trường làm việc* để agent không thể nấu hỏng món ăn [23:06].

Trụ cột 2: Khép kín Vòng lặp Kiểm chứng (Verification Loop) & Kỹ thuật `Feature Map`

Theo Lauren, kỹ năng quan trọng số một khi làm việc với Agent không phải là viết prompt hay viết spec, mà là **Verification (Kiểm chứng thực tế)** [06:07]. Kiểm chứng tự động chưa chắc giúp AI viết ra *code đẹp (good code)*, nhưng đảm bảo AI viết ra *code đúng (correct code)* — bước đệm bắt buộc để tiến tới tự động hóa [06:46].

Trong video, cô kể lại bài toán thực tế khi mới vào làm hiệu năng cho **Agents Window** của Cursor (tên mã nội bộ là **Glass** [09:30], viết bằng React/Electron):

1. **Nút thắt "Người chuyển lời" (Human Relay):** Ban đầu, cô tự mở Chrome DevTools, chụp Flame Graph, lấy CPU trace rồi gửi ảnh chụp màn hình hoặc lỗi console cho AI phân tích [08:06]. Cô nhận ra chính mình là cổ chai hiệu năng [08:50].
2. **Xây dựng Skill điều khiển (`control-glass`):** Cô đóng gói công cụ giao tiếp qua **Chrome DevTools Protocol (CDP)** (hoặc công cụ điều khiển iOS Simulator đối với ứng dụng di động) thành CLI/Skill để AI tự khởi chạy bản build, tự thao tác UI và tự lấy CPU trace/heap snapshot [09:51].
3. **Mảnh ghép quyết định mang tên `Feature Map` (chi tiết đắt giá trong video):**
    - Khi AI đã biết mở ứng dụng lên, nó vẫn hoàn toàn vô dụng vì... **không biết tính năng đó nằm ở đâu trên màn hình** [10:54]. Ví dụ: người dùng báo lỗi *"thanh sidebar bên trái bị lag"* hoặc *"tab PR không chạy"*, AI sẽ loay hoay mò mẫm trong code và click bừa trên UI rất tốn thời gian [10:47].
    - Lauren giải quyết bằng cách tạo ra file `Feature Map` [10:26]: một bản đồ cấu trúc mô tả toàn bộ tính năng dưới góc nhìn người dùng, đường dẫn điều hướng, phím tắt (keyboard shortcuts) và đặc biệt là các **DOM attributes / CDP selectors** tương ứng với từng vùng giao diện [13:05].
    - Nhờ `Feature Map`, ngay cả khi đồng nghiệp quăng vào kênh Slack một bức ảnh chụp màn hình cụ thể kèm đúng dấu chấm hỏi `"???"` [12:30], Agent (như bot đám mây **Benny** [25:22]) vẫn tự đối chiếu ảnh với `Feature Map`, tự mở máy ảo trên Cloud, tái hiện đúng lỗi và kiểm tra xem lỗi đó đã được sửa trên nhánh `main` hay chưa [25:55].

Trụ cột 3: Bộ công cụ `pstack` & Cơ chế `Eval Playbook` (Unit Test cho AI Agent)

Tên gọi **pstack** (*Poteto Stack*) bắt nguồn từ một trò đùa vui nhộn nhạo theo bộ công cụ `gstack` (*Garry Stack*) của Garry Tan — CEO Y Combinator [13:44]. Thay vì viết từ lý thuyết, `pstack` được xây dựng tích lũy từ những lần Lauren bắt quả tang AI làm ẩu [16:40]:

Thành phần trong `pstack`	Vấn đề thực tế của AI	Cơ chế giải quyết của Lauren Tan
Skill `/how` & `/why`	AI đoán mò nguyên nhân lỗi mà không thèm đọc các file code liên quan trong lịch sử gọi tool [15:06].	Bắt buộc Agent truy vết mã nguồn, lịch sử Git và chứng minh bằng dữ kiện runtime trước khi được phép đề xuất hướng sửa [16:52].
`create-verification-skill` & `maintain-verification-skill`	Agent không biết cách chạy thử ứng dụng hoặc tài liệu hướng dẫn kiểm thử bị lỗi thời khi sản phẩm thay đổi [17:15].	Tự động quét codebase để sinh `Feature Map` ban đầu [11:44] và tự động cập nhật bản đồ này khi giao diện thay đổi [13:20].
`/poteto-mode` (Router & 23 Playbooks)	Agent bỏ bước, làm tắt hoặc báo cáo khống kết quả khi xử lý tác vụ phức tạp.	Đọc danh mục 24 nguyên tắc (Principles), chọn đúng Playbook (Bug fix, Perf, Refactor...), chép nguyên văn các bước vào Todo list (nếu bỏ qua bước nào phải ghi rõ `skip: lý do`), và gắn nhãn từng kết luận là `measured` (đã đo đạc), `inferred` (suy luận) hay `guess` (phỏng đoán).
`Eval Playbook` (Kiểm thử Skill)	Làm sao biết khi sửa file Markdown của một Skill thì AI thông minh hơn hay ngốc đi? [19:33]	Coi **Eval như Unit Test dành cho Agent** [17:54].

Chi tiết kỹ thuật đặc sắc về cách Lauren chạy `Eval Playbook`:

- Cô dùng một **Coordinator Agent** để lập thang điểm đánh giá (rubric), sau đó sinh ra hàng loạt **Sub-agents** chạy trong các thư mục cô lập [19:03].
- **Mẹo chống "gian lận hành vi":** Các thư mục này phải được đặt tên ngụy trang khéo léo để Sub-agent **không nhận ra nó đang bị chấm điểm**, bởi cô phát hiện ra rằng khi LLM biết mình đang trong bài test đánh giá (eval), chúng sẽ tự động thay đổi hành vi khác với lúc làm việc thật [19:18]!
- **Chấm điểm chéo (Cross-model Judge) & Hill-climbing:** Cô dùng một mô hình thuộc họ khác làm giám khảo (Judge Agent) để tránh thiên kiến tự khen mình [21:39], đồng thời dùng lệnh `/loop` trong Cursor để Agent tự tinh chỉnh Skill lặp đi lặp lại cho đến khi đạt điểm tuyệt đối 10/10 [21:57].

Trụ cột 4: Chống "Organic Architecture" bằng Kiến trúc `Dune` & Tháp Kiểm soát 5 Tầng

Tại sao các dự án "vibe coding" ban đầu chạy rất nhanh nhưng càng về sau càng nát? Lauren gọi đó là **"Organic Architecture" (Kiến trúc mọc tự phát)** [32:57]. Bản chất của AI là luôn tìm con đường ngắn nhất, tiện nhất để hoàn thành yêu cầu trước mắt [33:12]. Phiên bản đầu tiên của Grok Bot được dựng mẫu cực nhanh mà không có rào chắn, dẫn đến những file khổng lồ hàng chục nghìn dòng code nơi AI cứ thế nhét thêm logic vào [32:46].

Lauren đã dành hơn **600 PR** [34:16] để đập đi xây lại Grok Bot trên một kiến trúc siêu nghiêm ngặt mang tên mã là **Dune** (*"Next.js dành cho ứng dụng Electron, được thiết kế chuyên biệt cho AI Agent viết code"*) [38:03].

Triết lý cốt lõi của **Dune** là: **"Biến con đường ngắn nhất thành con đường đúng đắn nhất" (The shortest path is the best path)** [43:32]. Cô phân chia hệ thống phòng thủ thành **2 nhóm tầng kiểm soát** rõ rệt [44:17]:

1. **Nhóm Ràng buộc Cứng (Hard Constraints — Làm đỏ CI ngay lập tức):**
    - **Cấu trúc thư mục đóng gói theo tính năng (Feature Collocation):** Mọi thành phần của một tính năng (`feature`, `entry points`, `transcript cards`) bắt buộc nằm gọn trong một thư mục duy nhất [42:56]. Agent không cần grep tìm kiếm khắp codebase; 80% công việc chỉ diễn ra cục bộ trong đúng thư mục đó [43:18].
    - **Kiểm soát đồ thị phụ thuộc (Dependency Graph Isolation):** Trong ứng dụng Electron, nếu để code tính toán nặng hoặc I/O lọt vào luồng giao diện (`renderer thread`), khung hình sẽ vượt quá ngân sách 16ms (mất mốc 60 FPS) gây giật lag [40:59]. Lauren tách cứng thư mục `electron-main` và `electron-renderer`, đồng thời cài CI kiểm tra cây `import`: bất kỳ lệnh import chéo nào vi phạm đều làm gãy build ngay lập tức [41:48].
    - **Cấm tuyệt đối các "Footguns" bằng Static Analysis / Linter:**
        - **Cấm `useEffect` trong React:** Mọi hành vi dùng `useEffect` đều bị CI chặn đứng để tránh lỗi vòng lặp render và rò rỉ bộ nhớ [38:26].
        - **Cấm viết Code Comments (`no-comments`):** Một quy tắc gây sốc nhưng cực kỳ thực tế — cô nhận thấy 99% comment do AI viết chỉ là ghi chú lịch sử vô nghĩa kiểu `// Lauren đã dặn không được làm thế này` [39:13] thay vì giải thích kiến trúc. Vì vậy, CI cấm hoàn toàn comment rác của AI [38:57].
2. **Nhóm Ràng buộc Mềm (Soft Constraints — Rules, `AGENTS.md`, Skills, Bugbot):**
    - Lauren nhấn mạnh: Agent hoàn toàn có thể "quên" hoặc phớt lờ nội dung trong `AGENTS.md` hay văn bản hướng dẫn [45:05]. Nếu một dự án chỉ dựa vào các file Rules và Style Guide mà thiếu đi tầng Static Analysis / Compiler / CI chặn cứng, sớm muộn codebase cũng sẽ biến thành đống rác [45:24].
    - **Quy tắc vàng chống "Code Review Land":** Mỗi khi con người phải gõ một dòng bình luận nhắc nhở AI trên PR, hãy coi đó là một "mùi hôi hệ thống" (anti-pattern) [46:43]. Thay vì nhắc AI trên PR, hãy biến lời nhắc đó thành một luật Linter, một bài kiểm tra CI hoặc thay đổi cấu trúc để lỗi đó **không còn khả năng tồn tại về mặt vật lý** [46:54].

Trụ cột 5: Vòng trong – Vòng ngoài, Chiến thuật Chia nhỏ PR & Kinh tế học Token (Token ROI)

Khi "Vòng trong" (nhận việc → sửa code → chạy kiểm thử thực tế → qua ải CI) đã đủ tin cậy, Lauren mở rộng sang "Vòng ngoài":

- **Chiến thuật Atomic PRs:** PR của cô dao động từ 50 đến 1.000 dòng code, nhưng cô luôn ép Agent tách công việc thành nhiều PR nhỏ mang tính nguyên tử (atomic) [36:59]. Nhờ vậy, lịch sử Git trở thành nguồn ngữ cảnh sạch và khi có lỗi phát sinh trên nhánh `main`, cô có thể `revert` chính xác từng mảnh nhỏ thay vì bơi trong một PR dài 40.000 dòng [37:25].
- **Gom nhóm nguyên nhân gốc (Root-cause Triage):** Thay vì cứ có báo cáo lỗi là giao ngay cho 1 Agent tạo bản vá (dễ sinh ra hàng chục bản vá chắp vá cho cùng một lỗi gốc), các Agent điều phối ("Chief of Staff") sẽ thu thập bằng chứng từ Slack, Linear, X vào tài liệu chung để rà soát điểm giao thoa trước khi quyết định sửa tận gốc.
- **Bài toán Kinh tế học Token (Token ROI):**
    - Trong video, Lauren thẳng thắn thừa nhận cô có lợi thế làm việc tại một phòng lab AI nên có lượng token gần như không giới hạn [48:04]. Việc dùng более 600 PR để tái cấu trúc Grok Bot và cho nhiều mô hình kiểm chứng chéo tiêu tốn lượng token khổng lồ ở giai đoạn đầu [48:36].
    - Tuy nhiên, dưới góc nhìn của một Kiến trúc sư hoặc Lãnh đạo Kỹ thuật, đó là bài toán **ROI (Tỷ suất hoàn vốn)** [48:28]: Thay vì tuyển hàng nghìn kỹ sư với chi phí lương đắt đỏ và gánh nặng quản lý cồng kềnh như các Big Tech truyền thống [48:57], việc đốt token một lần để xây dựng kiến trúc "chống đạn" giúp **mọi mô hình nhỏ/rẻ hơn** đều viết được code chất lượng cao [50:19], đồng thời giúp cả Product Manager, Designer và đội ngũ Go-To-Market tự gửi PR sửa lỗi và thêm tính năng trực tiếp qua giao diện chat của Grok Bot mà không sợ làm hỏng hệ thống [54:12].

3. Đánh giá Phản biện & Điểm giới hạn cần lưu ý

Bài học của Lauren Tan rất ấn tượng, nhưng khi áp dụng vào các hệ thống doanh nghiệp (như ngân hàng, tài chính, ERP, POS hay hạ tầng dữ liệu lớn), cần nhìn rõ **3 ranh giới kỹ thuật**:

1. **Quyết định Cửa hai chiều (Two-way doors) vs. Cửa một chiều (One-way doors):**
    - Grok Bot và Cursor Agents Window phần lớn là **ứng dụng Client/Desktop (React, Electron)** [07:36]. Một lỗi giao diện hoặc hiệu năng lọt lên `main` có thể được phát hiện qua lấy mẫu (sampling), `revert` trong vài giây và phát hành bản build mới mà không gây hậu quả vĩnh viễn [37:25].

- Ngược lại, với các thay đổi liên quan đến **Database Schema Migration, giao dịch thanh toán, bất biến tài chính (financial invariants) hoặc bảo mật phân quyền**, việc "merge trước, kiểm tra lấy mẫu sau" là rủi ro không thể chấp nhận. Chính Lauren cũng thừa nhận mức độ tự chủ của AI tỷ lệ thuận với **khả năng kiểm chứng tự động và khả năng đảo ngược hoàn toàn** của tác vụ đó.

2. **Sự dịch chuyển vai trò của Kỹ sư Phần mềm (Từ Coder sang Harness Architect):**
   - Giá trị cốt lõi của kỹ sư cấp cao không còn nằm ở tốc độ gõ code tính năng, mà nằm ở năng lực thiết kế **Kiến trúc Ràng buộc (Constraint Architecture)**: viết Custom AST Linter, thiết kế ranh giới Module/Package (như ArchUnit trong Java hay Dependency Cruiser/ESLint trong TypeScript), và xây dựng môi trường giả lập (Sandbox/Playwright E2E) để Agent tự kiểm chứng.
3. **Tách biệt rạch ròi giữa "Công cụ tất định" (Deterministic Tools) và "Phán đoán của AI" (LLM Judgment):**
   - Đừng để AI viết đi viết lại các đoạn script tạm bợ (disposable scripts) để test hay chuyển đổi code trong mỗi phiên chat. Những thao tác cơ học lặp lại phải được đóng gói thành CLI/Script cố định; chỉ dùng token của LLM cho việc đọc bằng chứng và ra quyết định kiến trúc.

http://googleusercontent.com/youtube_content/1 *Dữ liệu về những video bạn đã xem trên YouTube sẽ được lưu trữ trong Nhật ký hoạt động của bạn trên YouTube. Dữ liệu của bạn sẽ được YouTube lưu trữ và sử dụng theo Điều khoản dịch vụ của YouTube*

User prompt: Phân tích, nghiên cứu, tìm hiểu chi tiết tất cả vấn đề áp dụng triết lý của Lauren Tan vào hệ thống thực tế Ví dụ: Phân tích chi tiết cấu trúc 51 skills trong bộ pstack Thiết kế Tháp kiểm soát CI/Linter chống code rác cho dự án Cách xây dựng Feature Map và Verification Skill với Playwright ... ...

Response: Để chuyển hóa triết lý của Lauren Tan (`@poteto`) từ câu chuyện "1 người điều hành hơn 20 AI Agent, merge hơn 2.000 PR/tháng" thành một **kiến trúc kỹ thuật có thể vận hành trên hệ thống thực tế**, chúng ta cần bóc tách toàn bộ phần lõi kỹ thuật (under-the-hood mechanics) của bộ công cụ **`pstack`** và kiến trúc **`Dune`**.

Mục tiêu cốt lõi của Lauren Tan không phải là "tối đa hóa số dòng code" (maximize LOC), mà ngược lại: **viết ít code hơn, ưu tiên xóa code thừa, giữ mỗi PR ở mức nguyên tử (atomic) và ép mọi thay đổi phải đi qua kiểm chứng thực tế trên sản phẩm đang chạy (real artifact) trước khi merge**.

Bản nghiên cứu chuyên sâu dưới đây phân tích chi tiết **4 trụ cột kỹ thuật** để áp dụng trực tiếp vào các hệ thống sản xuất (Production Systems) từ Frontend, Backend Microservices cho đến hạ tầng kiểm thử tự động E2E.

PHẦN 1: Giải phẫu Kiến trúc `pstack` (Hệ sinh thái hơn 50 Skills, 23 Playbooks & 24 Principles)

Khi nhìn vào kho mã nguồn `pstack` (với khoảng 51–58 tệp `SKILL.md` tùy phiên bản port sang Claude Code, Codex, Pi hay OpenCode), nhiều kỹ sư lầm tưởng đây là một bộ sưu tập prompt rời rạc. Thực chất, `pstack` được thiết kế như một **Hệ điều hành Điều phối Agent 3 tầng (3-Layer Agent Orchestration Architecture)**:

1. **Tầng 1 – Bộ định tuyến trung tâm (Orchestrator):** Duy nhất 1 skill đầu vào là `/poteto-mode`.
2. **Tầng 2 – Kịch bản thực thi (23 Playbooks):** Các quy trình chuẩn hóa theo từng loại công việc (Bug fix, Feature, Perf, Refactoring, Investigation...).
3. **Tầng 3 – Kỹ năng chuyên biệt (~30 Workflow Leaf Skills) & Nguyên tắc kỹ thuật (24 Engineering Principles):** Các mô-đun lá được gọi theo nhu cầu của từng bước trong Playbook.

1.1. Ba cơ chế kỹ thuật "ăn tiền" nhất trong thiết kế của `pstack`

Trước khi xem danh sách các skill, cần nắm rõ 3 cơ chế kiến trúc giúp `pstack` chống lại hiện tượng **Agent làm ẩu (Slop)** và **Trôi ngữ cảnh (Context Drift)**:

Cơ chế 1: Chống "Tự ý bỏ bước" bằng Verbatim Todo-List Copy

Khi bạn giao việc qua `/poteto-mode`, bộ định tuyến không cho phép LLM tự tóm tắt quy trình hay tự nghĩ ra một kế hoạch ngắn hơn. Nó khớp yêu cầu với 1 trong 23 Playbooks và **sao chép nguyên văn (verbatim) từng bước của Playbook đó vào Todo List trước khi bắt đầu suy luận về bài toán**.

- Nếu một bước không cần thiết trong ngữ cảnh hiện tại, Agent **không được phép âm thầm xóa bước đó** mà phải giữ nguyên trên danh sách và ghi rõ lý do bỏ qua (ví dụ: `skip: không vượt qua ranh giới hàm nên không cần chạy /architect`).

Cơ chế 2: `disable-model-invocation: true` (Kiểm soát kích hoạt chặt chẽ)

Trong phần khai báo YAML frontmatter của các tệp `SKILL.md` thuộc nhóm Principles và phần lớn Workflow Skills, Lauren đặt cờ `disable-model-invocation: true`.

- **Tại sao cần cờ này?** Trong các Agent Harness hiện đại (Cursor, Claude Code, OpenCode), nếu để mặc định, LLM sẽ tự đọc phần `description` của hàng chục skill và tự ý nạp các quy tắc không liên quan vào ngữ cảnh chỉ vì trùng từ khóa, gây nhiễu loạn bộ nhớ (context pollution).
- Cờ này khóa cứng quyền tự kích hoạt: các Principle và Leaf Skill **chỉ được nạp vào bộ nhớ khi bước hiện tại trong Playbook gọi đích danh tên của chúng**.

Cơ chế 3: "Hai cánh cửa, Một căn phòng" (`poteto-agent` Wrapper)

Vấn đề lớn nhất khi một Agent cha (Coordinator) đẻ ra 5–10 Sub-agents con để làm việc song song là: **các Sub-agent con không có chung "văn hóa kỹ thuật" với Agent cha**, dẫn đến code viết ra mỗi nơi một kiểu.

- Lauren giải quyết bằng một Subagent Wrapper siêu nhỏ chỉ vỏn vẹn 6 dòng mang tên `subagent_type: "poteto-agent"`.
- Toàn bộ nội dung của `poteto-agent` chỉ là một lệnh bắt buộc: *"Đọc toàn bộ `poteto-mode/SKILL.md` bao gồm bảng chỉ mục 24 Nguyên tắc (Principles Index) trước khi thực hiện bất kỳ công việc nào"*.
- Nếu dùng `generalPurpose` subagent mặc định của công cụ, bước đọc này sẽ bị bỏ qua và Agent con sẽ viết code chệch hướng ngay lập tức. Nhờ `poteto-agent`, dù Agent cha chạy trên mô hình suy luận sâu (như Claude Opus) và đẻ ra 4 Agent con chạy trên mô hình tốc độ cao (như Grok hoặc Composer) để viết code, tất cả đều tuân thủ cùng một bộ quy chuẩn.

1.2. Ma trận Phân vai Đa mô hình (Multi-Model Role Routing)

Thông qua lệnh `/setup-pstack`, hệ thống tự động nhận diện các mô hình bạn có quyền truy cập và ghi cấu hình vào tệp rule (`~/.cursor/rules/pstack-models.mdc` hoặc `~/.claude/rules/pstack-models.md`) theo **4 vai trò độc lập**:

Vai trò (Role)	Bản chất công việc (Contract)	Nhiệm vụ điển hình trong `pstack`	Cách chọn Model tối ưu
`worker`	Nhanh, chi phí thấp, tuân thủ lệnh đơn giản.	Tìm kiếm mã nguồn (explorers), sửa đổi nhỏ, chạy tác vụ `/swarm` hàng loạt.	Mô hình tốc độ cao / Flash / Haiku / Composer-fast.
`builder`	Bám sát đặc tả tốt nhất, xử lý ngữ cảnh dài (long context).	Viết code tính năng, sửa bug, refactoring, tối ưu hiệu năng (`hillclimb`).	Grok 4.6, Claude Sonnet/Opus hoặc GPT-5 Codex.
`judge`	Suy luận sâu nhất (deepest reasoning), văn phong chuẩn xác.	Tổng hợp kiến trúc, điều phối chính, chấm điểm đánh giá chéo (cross-judge).	Claude Opus (Thinking xHigh).
`peer`	Mô hình suy luận mạnh nhưng **bắt buộc khác họ (different family)** với `judge`.	Tham gia hội đồng phản biện (`/interrogate`, `/architect`, `/arena`) để chống thiên kiến cùng một họ model.	Phối hợp chéo: Opus + Grok + Gemini/GPT.

> **Lưu ý thực tế:** Nếu môi trường của bạn chỉ có duy nhất 1 mô hình, `pstack` sẽ tự động gộp cả 4 vai trò về mô hình đó, nhưng biến các hội đồng phản biện (panels) thành **các lượt chạy độc lập trên ngữ cảnh mới hoàn toàn (fresh context)** để đảm bảo không bỏ qua bất kỳ cổng kiểm duyệt nào.

1.3. Danh mục 23 Playbooks (Kịch bản Thực thi Chuẩn hóa)

Thay vì dùng chung một quy trình cho mọi việc, `/poteto-mode` định tuyến yêu cầu vào đúng 1 trong 23 Playbooks chuyên biệt sau:

Nhóm mục tiêu	Tên Playbook	Quy trình kỹ thuật bắt buộc (Execution Contract)
1. Thấu hiểu trước khi chạm vào code (Read-only)	**1.** `Investigation`	Chỉ đọc mã nguồn (Read-only). Gọi `/how` để lần theo luồng thực thi và `/why` để truy vết lịch sử trước khi trả lời câu hỏi *"Hệ thống đang chạy thế nào / Chúng ta có chắc không?"*.
	2. `Runtime forensics`	Chẩn đoán triệu chứng đang chạy thực tế (rò rỉ bộ nhớ, CPU quay vòng khi nghỉ, giật khung hình) bằng công cụ đo đạc (instrumentation); dừng lại ở kết luận chẩn đoán, chưa sửa code.
	3. `Trace forensics`	Phân tích tệp dữ liệu đo đạc đã thu thập sẵn (`cpuprofile`, `trace`, `spindump`, `heap snapshot`), chuyển dữ liệu lớn thành dạng truy vấn được, chỉ ra đúng hàm/frame gây nghẽn.
2. Sửa lỗi & Tối ưu có phương pháp khoa học	**4.** `Bug fix`	**Repro-first:** Bắt buộc tái hiện được lỗi trước khi sửa. Đưa ra các giả thuyết cạnh tranh, loại trừ bằng bằng chứng runtime, viết bài test báo đỏ (`/tdd`), sửa lỗi và kiểm chứng lại trên đúng bề mặt ban đầu.
	5. `Perf`	Đo đạc điểm nghẽn có số liệu cụ thể (baseline), thực hiện cải tiến và chứng minh bằng chỉ số đo đạc sau khi sửa. Tuyệt đối không chấp nhận kết luận cảm tính *"cảm giác nhanh hơn"*.
	6. `Hillclimb`	Tối ưu hóa liên tục một chỉ số (metric) hướng tới ngưỡng mục tiêu: Lặp qua từng giả thuyết → Đo trước/sau → Nếu tốt hơn thì `git commit` giữ lại, nếu tệ hơn thì `discard` hủy bỏ → Ghi vào bảng log.
3. Xây dựng & Thay đổi cấu trúc	**7.** `Feature`	Xây dựng tính năng mới bắt đầu từ việc định hình **cấu trúc dữ liệu (named data shape)**, chạy `/how` để hiểu vùng code xung quanh, chạy `/architect` nếu vượt qua ranh giới module, giao việc cho `builder`, phản biện bằng `/interrogate` và chạy kiểm chứng thực tế.
	8. `Refactoring`	Thay đổi cấu trúc nhưng **bảo toàn 100% hành vi (behavior-preserving)**. Bắt buộc tạo bài kiểm tra đặc tả hành vi hiện tại (characterization test / snapshot / equivalence script) trước, sau đó biến đổi từng bước nhỏ giữ cho test luôn xanh.

Nhóm mục tiêu	Tên Playbook	Quy trình kỹ thuật bắt buộc (Execution Contract)
	9. `Prototype`	Xây dựng bản nháp dùng một lần (throwaway sketch) nhanh và rẻ nhất để kiểm chứng một quyết định thiết kế hoặc giải quyết tranh cãi kỹ thuật bằng thực nghiệm.
	10. `Visual parity`	Đảm bảo độ tương đồng giao diện chính xác tới từng điểm ảnh (pixel-exact UI equivalence) giữa hai bản cài đặt; bất kỳ sai lệch pixel nào khác 0 đều tính là thất bại.
	11. `Multi-phase plan`	Chia nhỏ công việc lớn kéo dài qua nhiều giai đoạn hoặc chuỗi PR xếp chồng (stacked PRs), trong đó mỗi giai đoạn kết thúc bằng một bài kiểm chứng độc lập.
4. Duyệt, Đóng gói & Merge PR	**12.** `Opening a PR`	Playbook nội bộ được gọi ở cuối mọi Playbook viết code: gom các commit nhỏ theo thứ tự hợp lý, đặt tiêu đề theo chuẩn Conventional Commits và viết mô tả PR ngắn gọn dạng bản tin kỹ thuật.
	13. `Babysit`	Chăm sóc một PR hoặc một chuỗi PR (stack) cho đến khi sẵn sàng merge: kiểm tra xung đột (conflicts) trước, báo cáo nếu cần rebase, xử lý các luồng bình luận review và sửa lỗi CI.
	14. `Shipping`	Tách biệt hoàn toàn với `Babysit` ("CI xanh chưa chắc đã an toàn để merge"). Dùng một Agent mới hoàn toàn để kiểm chứng độc lập từng PR trong stack, kiểm tra xem kết luận duyệt cũ còn khớp với commit hiện tại không, và chỉ merge chuỗi PR liên tục đã được xác thực từ gốc lên.
	15. `Autopilot-full`	Vận hành hàng đợi các PR độc lập đi thẳng tới trạng thái `merged` với một Agent phụ trách riêng từng PR; một đàn Agent (`/swarm`) kiểm chứng từng bản vá mới.
	16. `Autopilot-stack`	Xây dựng và kiểm chứng một chuỗi PR tuyến tính (linear stack) hoàn chỉnh để con người xem xét lần cuối và bấm nút land.
5. Tự hành dài hạn & Chuyển giao phiên (Autonomy)	**17.** `Autonomous run`	Chạy liên tục một tác vụ dài hạn không dừng nghỉ (thường kết hợp với `/loop` khi kỹ sư đi ngủ) cho đến khi điều kiện hoàn thành (pass/fail check) được thỏa mãn, kèm nhật ký quyết định.
	18. `Orchestrate`	Dành cho dự án quy mô nhiều ngày với hàng chục PR xếp chồng: một Coordinator Chat cố định chỉ huy cả một hạm đội Sub-agents.
	19. `Session pickup`	Tiếp quản công việc đang dang dở của một Agent trước đó bằng cách đọc lại transcript, nhánh Git và file nhật ký quyết định để biết chính xác việc gì đã xong, tránh làm lại từ đầu.
	20. `Pause safely`	Dừng công việc đang chạy ở một ranh giới nguyên tử (atomic boundary) an toàn, lưu trữ trạng thái bền vững và viết ghi chú bàn giao (`resume note`) cho phiên sau.
6. Công cụ & Hạ tầng Agent	**21.** `Authoring a skill`	Quy trình thiết kế, viết mới hoặc chỉnh sửa tệp `SKILL.md` đảm bảo chuẩn cấu trúc và kích hoạt đúng ngữ cảnh.
	22. `Eval`	Kiểm thử mù (blinded test) xem việc thay đổi nội dung một Skill hoặc Prompt ảnh hưởng thế nào đến hành vi thực tế của Agent trên nhiều model.
	23. `Worktree cleanup`	Thu hồi dung lượng ổ cứng bằng cách phân loại mọi Git worktree theo trạng thái merge, code chưa commit và phiên chat đang mở; chỉ xóa những worktree đã an toàn tuyệt đối và dừng lại hỏi con người nếu còn code dở dang.

1.4. Giải phẫu 24 Nguyên tắc Kỹ thuật (The 24 Engineering Principles)

Ở dưới đáy của kiến trúc `pstack` là 24 tệp kỹ năng nguyên tắc (`principle-*`). Điều đặc biệt là `poteto-mode` ép Agent: **"Phải nêu tên nguyên tắc đã định hình nên quyết định của bạn, và chỉ rõ nó đã làm thay đổi lựa chọn nào. Việc trích dẫn tên nguyên tắc mà không gắn với một quyết định cụ thể còn tệ hơn không trích dẫn"**.

Dưới đây là bảng phân tích các nguyên tắc cốt lõi chia theo 5 nhóm kiến trúc:

Nhóm	Tên Principle Skill	Quy tắc cốt lõi & Cách áp dụng vào Codebase
1. Core (Tư duy nền tảng & Tối giản hóa)	`principle-laziness-protocol`	**Giao thức "Lười biếng" (Chống Over-engineering):** Viết code đối với AI là quá rẻ nên AI rất thích đẻ thêm tầng trừu tượng. Hãy mượn "sự mệt mỏi của người bảo trì": ưu tiên **xóa code** hơn thêm code; tạo diff nhỏ nhất giải quyết được vấn đề; nếu để trả lời một câu hỏi phải lần qua hơn 3 file/tầng thì phải làm phẳng (flatten) cấu trúc gọi hàm; gom các quyết định lặp lại về một nguồn duy nhất.
	`principle-foundational-thinking`	**Tư duy Nền tảng:** Chọn đúng cấu trúc dữ liệu và kiểu cốt lõi (core types) trước khi viết logic xử lý; xây dựng phần khung (CI, linter, hạ tầng test, shared types) trước khi làm tính năng; tự hỏi *"nếu một tác nhân khác sửa đổi trạng thái này đồng thời thì điều gì xảy ra?"* trước khi chia sẻ trạng thái.
	`principle-subtract-before-you-add`	**Trừ trước khi Cộng:** Trước khi thêm tính năng, refactor hay viết lại, hãy dọn dẹp code chết (dead code), các bộ kiểm tra dư thừa (redundant validators) và các stub cũ trước, rồi mới xây trên nền móng đã làm sạch.

Nhóm	Tên Principle Skill	Quy tắc cốt lõi & Cách áp dụng vào Codebase
	`principle-minimize-reader-load`	**Giảm thiểu Tải nhận thức của Người đọc:** Đo độ phức tạp trên 2 trục độc lập: (1) *Số tầng trung gian phải lần theo* và (2) *Lượng trạng thái ẩn/khả biến (mutable state) người đọc phải nhớ trong đầu*. Xóa bỏ các hàm bọc chỉ có 1 nơi gọi (one-caller wrappers) và các tầng chỉ truyền dữ liệu đi qua (pass-through layers). Bài kiểm tra: *Người đọc mới có trả lời được "Dữ liệu X đến từ đâu?" và "Cái gì có thể làm thay đổi X?" trong dưới 30 giây không?*
	`principle-redesign-from-first-principles`	**Thiết kế lại từ Nguyên lý gốc:** Khi tích hợp một yêu cầu mới vào thiết kế cũ, hãy thiết kế lại như thể yêu cầu đó đã là giả định nền tảng từ ngày đầu tiên, thay vì chắp vá (bolting it on) bằng các câu lệnh `if/else` đặc thù.
	`principle-outcome-oriented-execution`	**Thực thi hướng Kết quả cuối:** Trong các đợt di trú (migration) hoặc viết lại có nhiều giai đoạn, phải hội tụ thẳng về kiến trúc đích thay vì giữ lại các tầng tương thích tạm bợ (throwaway compatibility states).
	`principle-experience-first`	**Trải nghiệm Người dùng trên hết:** Khi phải đánh đổi giữa sự tiện lợi lúc lập trình và trải nghiệm người dùng (UX/Product), luôn chọn sự hài lòng của người dùng; thà giao ít tính năng nhưng mượt mà còn hơn nhiều tính năng thô kệch.
	`principle-exhaust-the-design-space`	**Vét cạn Không gian Thiết kế:** Khi đối mặt với tương tác UI mới hoặc quyết định kiến trúc chưa có tiền lệ trong repo, hãy dựng 2–3 bản mẫu (prototypes) cạnh tranh nhau và đặt cạnh nhau so sánh trước khi chốt.
	`principle-build-the-lever`	**Xây dựng Đòn bẩy (Công cụ):** Với mọi công việc không tầm thường, đừng sửa thủ công từng chỗ; hãy viết công cụ tự động hóa hoặc chứng minh nó (codemod, script chuyển đổi, bộ sinh code). Chính công cụ đó là hiện vật để người duyệt chạy lại.
2. Architecture (Kiến trúc & Ranh giới)	`principle-model-the-domain`	**Mô hình hóa Miền nghiệp vụ:** Khi viết logic có trạng thái phức tạp hoặc lặp lại giả định cấu trúc ở nhiều file, hãy mã hóa miền nghiệp vụ vào cấu trúc dữ liệu/state machine thay vì rải các câu lệnh điều kiện (`if/switch`) khắp nơi.
	`principle-boundary-discipline`	**Kỷ luật Ranh giới Hệ thống:** Đặt toàn bộ việc kiểm tra hợp lệ (validation), napsing dữ liệu và xử lý lỗi ở **ranh giới hệ thống** (CLI, cấu hình, network, API bên ngoài); bên trong ranh giới đó, hãy tin tưởng các kiểu dữ liệu nội bộ (internal types) và giữ cho logic nghiệp vụ là các hàm thuần khiết (pure functions).
	`principle-type-system-discipline`	**Kỷ luật Hệ thống Kiểu:** Khi thiết kế chữ ký hàm hoặc kiểu dữ liệu, hãy biến các **trạng thái bất hợp lệ thành không thể biểu diễn được (make illegal states unrepresentable)**, dùng Branded Types cho các kiểu nguyên thủy (ví dụ phân biệt `UserId` và `OrderId` dù cùng là `string`/`UUID`).
	`principle-make-operations-idempotent`	**Đảm bảo Tính Lũy đẳng (Idempotency):** Mọi lệnh, bước khởi tạo vòng đời hoặc vòng lặp xử lý đều phải hội tụ về đúng một trạng thái đích duy nhất bất kể bị sập giữa chừng, khởi động lại hay chạy lặp lại (retry).
	`principle-migrate-callers-then-delete-legacy-apis`	**Di trú toàn bộ nơi gọi rồi Xóa ngay API cũ:** Khi giới thiệu một API nội bộ mới, phải chuyển đổi toàn bộ các nơi đang gọi (callers) và xóa bỏ API cũ trong cùng một đợt sóng (same wave), tuyệt đối không nuôi song song hai hệ thống API nội bộ.
	`principle-separate-before-serializing-shared-state`	**Cô lập trước khi Tuần tự hóa Trạng thái chung:** Khi nhiều tác nhân đồng thời (threads, jobs, subagents) cùng ghi vào một file, một nhánh Git, một Redis key hoặc một bản ghi, giải pháp đầu tiên không phải là thêm khóa (lock/mutex) mà là **loại bỏ việc dùng chung trước** (mỗi tác nhân một worktree/key riêng); chỉ tuần tự hóa khi việc chia sẻ là bắt buộc về mặt nghiệp vụ.
3. Verification (Kiểm chứng & Gỡ lỗi)	`principle-prove-it-works`	**Chứng minh bằng Hiện vật Thực tế:** Không bao giờ báo cáo "đã xong" chỉ vì code biên dịch thành công ("it compiles") hoặc bài unit test giả lập (mock) báo xanh. Phải chạy trên sản phẩm thực tế và thu thập bằng chứng sống.
	`principle-fix-root-causes`	**Sửa tận Nguyên nhân Gốc:** Tái hiện lỗi trước, truy vấn "tại sao" cho đến khi chạm tới gốc rễ; **nghiêm cấm việc thêm các lệnh chặn `if (x == null) return` (nil-check guards) chỉ để bịt miệng lỗi crash mà làm che khuất dữ liệu hỏng bên dưới**.
	`principle-attack-the-premise`	**Tấn công vào Giả định Nền tảng:** Khi đã thử sửa 2–3 lần dựa trên cùng một giả định mà vẫn thất bại, hãy dừng ngay việc thử bản vá tiếp theo. Hãy phát biểu rõ giả định đang dùng và thiết kế một quan sát thực nghiệm để bác bỏ chính giả định đó.
	`principle-test-behavior-not-implementation`	**Kiểm thử Hành vi, không kiểm thử Cài đặt nội bộ:** Mỗi bài test viết ra phải chỉ rõ nó bắt được lỗi thực tế nào (defect detection), kiểm tra kết quả đầu ra hoặc hiệu ứng quan sát được thay vì kiểm tra các hàm private hay mock nội bộ.
	`principle-sequence-verifiable-units`	**Chia nhỏ thành các Đơn vị Kiểm chứng được:** Mọi kế hoạch nhiều bước hoặc chuỗi PR phải được cắt lát sao cho mỗi bước kết thúc bằng một bài kiểm tra độc lập chạy được ngay.
4. Delegation (Ủy quyền & Quản trị Agent)	`principle-guard-the-context-window`	**Bảo vệ Cửa sổ Ngữ cảnh:** Bộ nhớ ngữ cảnh của luồng điều phối chính (Main Thread) là tài nguyên quý giá nhất. Đẩy toàn bộ việc đọc file dài, phân tích log lớn, chạy tìm kiếm diện rộng sang các Sub-agents; chỉ trả bản tóm tắt cô đọng về luồng chính.

Nhóm	Tên Principle Skill	Quy tắc cốt lõi & Cách áp dụng vào Codebase
	`principle-never-block-on-the-human`	**Không bao giờ Đứng chờ Con người ở việc Đảo ngược được:** Khi Agent định dừng lại hỏi *"Tôi có nên làm theo cách X không?"*, nếu câu trả lời có thể kiểm chứng bằng cách chạy thử code hoặc nếu hành động đó dễ dàng đảo ngược (reversible), Agent phải **tự làm luôn, trình bày kết quả kèm bằng chứng** và để con người hiệu chỉnh sau. Chỉ dừng lại xin phép trước các hành động không thể đảo ngược (xóa dữ liệu, force-push nhánh chung, gọi API bên thứ ba mất phí).
5. Meta (Tự tiến hóa hệ thống)	`principle-encode-lessons-in-structure`	**Mã hóa Bài học vào Cấu trúc:** Khi bạn bắt gặp mình phải nhắc lại cùng một chỉ dẫn lần thứ hai, đừng viết thêm chữ vào tài liệu hay prompt! Hãy mã hóa bài học đó theo thứ tự ưu tiên từ cao xuống thấp: **Kiến trúc thư mục → Hệ thống kiểu (Types) → Linter (với thông báo lỗi chỉ rõ cách sửa) → CI check → Bài test → và cuối cùng mới là Docs**.

1.5. Nhóm Workflow Skills chuyên biệt đáng chú ý nhất

Bên cạnh Playbooks và Principles, `pstack` cung cấp các công cụ lá (Leaf Skills) giải quyết những bài toán rất cụ thể trong đời sống kỹ sư:

1. **`/how` vs `/why` vs `/teach`:**
   - `/how`: Chia 2–4 Sub-agents đọc song song (Data model, Request path, Config/Metrics) rồi tổng hợp thành mô hình hoạt động hiện tại của hệ thống, kèm chế độ Critic kiểm tra theo `boundary-discipline`.
   - `/why`: Tự động phát hiện toàn bộ các nguồn **MCP (Model Context Protocol)** đang kết nối ở thời điểm chạy và truy vấn song song tới **7 nguồn bằng chứng lịch sử**: Source control (`git log/blame`), Issue tracker (Linear/Jira), Tài liệu thiết kế (RFCs/ADRs), Chat thời gian thực (Slack), Hạ tầng giám sát (Observability), Hệ thống theo dõi lỗi (Sentry/Datadog) và Kho phân tích (Analytics Warehouse) để giải thích **tại sao một đoạn code kỳ lạ lại được viết như vậy**.
   - `/teach`: Kết hợp cả `/how` và `/why` để giảng giải toàn diện một phân hệ từ cơ chế đến lịch sử qua chuỗi sơ đồ trực quan.
2. **`/architect` vs `/arena` vs `/swarm` vs `/interrogate` (Bộ tứ Đa tác nhân):**
   - `/architect`: Khi code chuẩn bị vượt qua ranh giới một hàm/module, gọi nhiều mô hình thiết kế từ **góc nhìn của nơi gọi (caller-first usage)**, chốt kiểu dữ liệu và hình thái module trước khi cho phép viết code.
   - `/arena`: Cho N Agent chạy đua giải quyết **cùng một bài toán** theo các cách tiếp cận khác nhau trong các worktree riêng biệt, sau đó chọn bản tốt nhất làm gốc và "ghép cành" (graft) những ý tưởng hay từ các bản thua cuộc vào.
   - `/swarm`: Khác với `/arena` (cùng làm 1 việc để so sánh), `/swarm` chia nhỏ công việc thành N lát cắt khác nhau (slices) cho N worker chạy song song rồi gom về một báo cáo tổng hợp duy nhất.
   - `/interrogate`: Hội đồng phản biện đối kháng gồm 4 Reviewer chạy trên các họ mô hình khác nhau cố gắng tìm mọi cách "bẻ gãy" bản diff của bạn trước khi mở PR.
3. **`/blast-radius`:** Khi bạn có một thay đổi trông có vẻ nhỏ, skill này quét toàn bộ hệ thống bên ngoài bản diff để tìm xem cái gì có thể bị vỡ, và **bắt buộc chứng minh lý do an toàn bằng cách chạy code thực tế** chứ không chấp nhận lập luận miệng.
4. **`/no-comments` & Subagent `comment-sicko`:** Trước khi review, gọi subagent chuyên biệt tên là `comment-sicko` để quét và lột sạch các comment giải thích thừa thãi do AI sinh ra. Đặc biệt, nếu phát hiện một comment chứa ràng buộc quan trọng kiểu `// KHÔNG ĐƯỢC XÓA DÒNG NÀY VÌ SẼ LỖI X`, nó sẽ đề xuất **biến câu comment đó thành một Type, một Runtime Assert, một Unit Test hoặc một luật Lint** rồi xóa comment đi!
5. **`/correct` & `/automate-me`:**
   - `/correct`: Đào lại lịch sử chat và lịch sử sửa PR để tìm các nhóm lỗi mà Agent cứ lặp đi lặp lại trong repo của bạn, sau đó tự động nâng cấp hệ thống phòng thủ ở tầng cao nhất có thể (Architecture → Types → Lint → CI → Test) và duy trì một bảng ánh xạ giữa "Quy tắc" và "Công cụ cưỡng chế quy tắc đó".
   - `/automate-me`: Quét lịch sử các phiên làm việc (transcripts) gần đây của chính bạn để học thói quen, khẩu vị kỹ thuật và tự sinh ra một `<tên-bạn>-mode` riêng chạy trên nền `pstack`.

PHẦN 2: Thiết kế "Tháp Kiểm soát 5 Tầng" (CI / Linter / Architecture Constraints) Chống Code Rác

Trong video chia sẻ về kiến trúc **Dune** của Grok Bot, Lauren Tan khẳng định một chân lý xương máu: **Nếu bạn chỉ kiểm soát AI bằng các tệp văn bản (`AGENTS.md`, `CLAUDE.md`, Rules, Style Guides), chắc chắn 100% codebase của bạn sẽ nát theo thời gian**, bởi vì LLM hoàn toàn có thể quên hoặc phớt lờ lời dặn khi ngữ cảnh dài ra.

Để 20+ Agent có thể tự động merge code mà con người không cần đọc từng dòng trước khi merge, hệ thống phải được bảo vệ bởi **Tháp Kiểm soát 5 Tầng** (đi từ cưỡng chế vật lý/biên dịch xuống tới hướng dẫn mềm):

```
================================================================================
TẦNG 1: KIẾN TRÚC THƯ MỤC & ĐỒ THỊ PHỤ THUỘC (Physical & Dependency Graph)
  → Feature Collocation + Chặn import chéo giữa các tầng/luồng (Làm đỏ CI)
--------------------------------------------------------------------------------
TẦNG 2: HỆ THỐNG KIỂU & TRÌNH BIÊN DỊCH (Type System & Compiler Diagnostics)
```

```
  → Branded Types, Illegal States Unrepresentable, Strict Null/Exhaustive Checks
--------------------------------------------------------------------------------
TẦNG 3: PHÂN TÍCH TĨNH & CUSTOM AST LINTERS (Static Analysis & Anti-Footguns)
  → Cấm hàm nguy hiểm, cấm comment rác của AI, thông báo lỗi chứa sẵn cách sửa
--------------------------------------------------------------------------------
TẦNG 4: CỔNG KIỂM CHỨNG ĐỘNG & E2E HARNESS (Automated Runtime Verification)
  → Playwright/CDP Feature Map, Testcontainers, Benchmark Regression Gate
--------------------------------------------------------------------------------
TẦNG 5: RÀNG BUỘC MỀM & ĐỐI KHÁNG ĐA MÔ HÌNH (Soft Rules & Adversarial Review)
  → AGENTS.md, pstack Playbooks, Bugbot, /interrogate (Cross-Model Review)
================================================================================
```

Dưới đây là cách thiết kế cụ thể Tầng 1, 2 và 3 cho cả **Frontend/TypeScript** và **Backend Enterprise (Java/Spring Boot)**.

2.1. Áp dụng cho Hệ thống Frontend / TypeScript / Node.js (Kiến trúc kiểu `Dune`)

A. Đóng gói theo Tính năng (Feature Collocation) & Khóa đồ thị phụ thuộc (`dependency-cruiser`)

AI Agent rất giỏi sao chép mẫu có sẵn ở ngay cạnh file nó đang sửa (local pattern matching), nhưng rất kém khi phải nhảy qua 7–8 thư mục ngang (`controllers/`, `services/`, `utils/`, `types/`, `views/`) để ghép một tính năng.

- **Thiết kế thư mục:** Gom toàn bộ code của một tính năng (`feature`, `entry-points`, `components`, `state`, `api-client`) vào một thư mục khép kín `src/features/<feature-name>/`.
- **Cưỡng chế ranh giới bằng `dependency-cruiser` trên CI:** Ngăn chặn tuyệt đối việc code giao diện gọi thẳng vào tầng nặng hoặc các feature tự ý chọc vào ruột của nhau mà không qua `index.ts` công khai.

```
// .dependency-cruiser.cjs — Cưỡng chế đồ thị phụ thuộc làm đỏ CI ngay lập tức
module.exports = {
  forbidden: [
    {
      name: 'no-cross-process-imports',
      comment: 'CẤM import code từ main/worker thread vào renderer/UI thread gây tụt FPS (vượt ngưỡng 16ms/frame). Hãy g
      severity: 'error',
      from: { path: '^src/renderer/' },
      to: { path: '^src/(main|worker|infra-heavy)/' }
    },
    {
      name: 'feature-encapsulation-boundary',
      comment: 'CẤM một feature import trực tiếp file nội bộ của feature khác. Chỉ được phép import qua public entry poi
      severity: 'error',
      from: { path: '^src/features/([^/]+)/' },
      to: {
        path: '^src/features/([^/]+)/.+',
        pathNot: '^src/features/$1/|^src/features/([^/]+)/index\\.ts$'
      }
    }
  ]
};
```

B. Thiết kế Linter "Dạy cho AI cách sửa ngay trong thông báo lỗi"

Theo nguyên tắc `encode-lessons-in-structure`, một luật Linter tốt dành cho AI Agent không chỉ báo *"Cấm dùng X"*, mà **chuỗi thông báo lỗi (`message`) phải viết sẵn hướng dẫn thay thế** để khi Agent đọc log lỗi từ terminal, nó biết sửa ngay trong lượt tiếp theo mà không cần hỏi con người:

```
// eslint.config.mjs — Chặn các "Footguns" mà AI hay lạm dụng
export default [
  {
    rules: {
      'no-restricted-syntax': [
        'error',
        {
          selector: "CallExpression[callee.name='useEffect']",
          message: "BANNED FOOTGUN: useEffect bị cấm trong kiến trúc này vì gây vòng lặp render và lỗi đồng bộ trạng thá
        },
        {
          selector: "TSAsExpression > TSAnyKeyword, TSAsExpression > TSUnknownKeyword",
          message: "BANNED TYPE ESCAPE: Cấm ép kiểu 'as any' hoặc 'as unknown'. Hãy parse dữ liệu tại ranh giới hệ thống
        }
      ],
      // Kết hợp với quy tắc cấm comment rác hoặc ép chạy /no-comments
      'no-warning-comments': [
        'error',
```

```
        {
          terms: ['todo', 'fixme', 'hack', 'temporary'],
          location: 'anywhere'
        }
      ]
    }
  }
];
```

2.2. Áp dụng cho Hệ thống Backend Enterprise (Java 25 / Spring Boot / CQRS / Messaging)

Trong môi trường Backend doanh nghiệp (như kiến trúc Microservices hoặc Modular Monolith sử dụng Spring Boot, PostgreSQL Read/Write Replica, Redis, Kafka/RabbitMQ), việc để AI Agent tự do viết code rất dễ dẫn đến các lỗi thảm họa: **N+1 Query ngầm, gọi HTTP/Kafka bên trong `@Transactional` làm treo Connection Pool, lộ Entity ra tầng API, hoặc ghi nhầm vào Read-Replica**.

Chúng ta có thể biến toàn bộ tiêu chuẩn của Solutions Architect thành **Hard Constraints** làm đỏ build ngay lập tức bằng **ArchUnit** kết hợp với **ErrorProne / Checkstyle**:

```
package vn.architecture.guardrails;

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.RestController;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.methods;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

@AnalyzeClasses(packages = "com.enterprise.platform", importOptions = ImportOption.DoNotIncludeTests.class)
public class AgentArchitectureHardConstraintsTest {

    // 1. Cưỡng chế Logical CQRS: Tách biệt tuyệt đối Command (Write) và Query (Read-Replica)
    @ArchTest
    static final ArchRule cqrs_query_handlers_must_be_read_only_and_never_touch_write_repositories =
        noClasses()
            .that().resideInAPackage("..query..")
            .should().dependOnClassesThat().resideInAPackage("..command..")
            .orShould().CallMethodWhere(target -> target.getName().startsWith("save")
                || target.getName().startsWith("delete")
                || target.getName().startsWith("update"))
            .because("VIOLATION [Boundary-Discipline & CQRS]: Query side chỉ được phép đọc từ Read-Replica/Projection. "
                + "Cách sửa: Chuyển mọi thao tác biến đổi dữ liệu sang CommandHandler trong package '..command..'.");

    // 2. Chống cạn kiệt Database Connection Pool: Cấm gọi Kafka/RabbitMQ/FeignClient từ tầng giữ DB Transaction
    @ArchTest
    static final ArchRule transactional_services_must_not_call_external_network_or_brokers_directly =
        noClasses()
            .that().areAnnotatedWith(Transactional.class)
            .should().dependOnClassesThat().resideInAnyPackage(
                "..infrastructure.client..",
                "org.springframework.kafka.core..",
                "org.springframework.amqp.rabbit.core.."
            )
            .because("VIOLATION [Foundational-Thinking]: Gọi mạng hoặc đẩy message vào Kafka/RabbitMQ bên trong @Transac
                + "sẽ giữ kết nối PostgreSQL khi chờ I/O mạng, gây sập Connection Pool. "
                + "Cách sửa: Ghi sự kiện vào bảng Outbox trong cùng Transaction hoặc phát Domain Event sau pha AFTER_COM

    // 3. Cưỡng chế Boundary Discipline: Cấm trả thẳng JPA Entity ra ngoài RestController
    @ArchTest
    static final ArchRule controllers_must_never_expose_persistence_entities =
        methods()
            .that().areDeclaredInClassesThat().areAnnotatedWith(RestController.class)
            .should().notHaveRawReturnType(type -> type.getPackageName().contains(".domain.entity")
                || type.getPackageName().contains(".persistence.entity"))
            .because("VIOLATION [Boundary-Discipline]: Không được trả trực tiếp Persistence Entity ra ngoài API boundary
                + "Cách sửa: Ánh xạ sang Java Record Response DTO tại ranh giới Controller/Application.");

    // 4. Feature Collocation & Module Encapsulation
    @ArchTest
    static final ArchRule internal_feature_packages_must_not_be_accessed_from_outside =
        classes()
```

```
            .that().resideInAPackage("..modules.(*).internal..")
            .should().onlyBeAccessed().byClassesThat().resideInAPackage("..modules.$1..")
            .because("VIOLATION [Minimize-Reader-Load]: Các class trong package '.internal.' là chi tiết cài đặt riêng c
                + "Cách sửa: Chỉ giao tiếp giữa các module thông qua API interface công khai tại '..modules.<name>.api..
}
```

> **Hiệu quả thực tế:** Khi AI Agent viết một hàm sửa lỗi và vô tình gọi `kafkaTemplate.send()` bên trong một hàm `@Transactional`, lệnh `./gradlew test` sẽ thất bại ngay lập tức kèm theo đúng thông báo `Cách sửa: Ghi sự kiện vào bảng Outbox...`. Agent tự đọc lỗi và tự sửa kiến trúc mà Solutions Architect không cần tốn một giây nào để comment trên PR!

PHẦN 3: Xây dựng `Verification Skill` và `Feature Map` Tự động với Playwright & CDP

Trong triết lý của Lauren Tan, **Verification Skill là ranh giới phân định giữa "đồ chơi" và "hệ thống sản xuất"**. Dựa trên mã nguồn thực tế của `create-verification-skill/SKILL.md` và `maintain-verification-skill/SKILL.md` trong `pstack`, dưới đây là kiến trúc chuẩn để xây dựng bộ kỹ năng kiểm chứng tự động cho dự án của bạn.

3.1. Cấu trúc chuẩn của một Verification Skill (`.cursor/skills/verify-<app>/` hoặc `.claude/skills/verify-<app>/`)

Một Verification Skill chuẩn không được phép chứa ví dụ giả (placeholders) mà phải khảo sát trực tiếp từ mã nguồn của dự án ("Interview the repo, not the user") và bao gồm **6 phần bắt buộc** cộng với thư mục `features/` **(Feature Map)**:

```
.claude/skills/verify-app/          (hoặc .cursor/skills/verify-app/)
├── SKILL.md                        # Chứa 6 mục: Launch, Doctor, Drive, Evidence, Cleanup, Helpers
├── features/                       # FEATURE MAP: Bản đồ định vị tính năng cho AI Agent
│   ├── README.md                   # Chỉ mục tất cả tính năng + Danh sách Unmapped routes (có lý do)
│   ├── auth-and-session.md         # Đường dẫn, trạng thái, DOM/API selectors, gotchas
│   ├── order-checkout.md           # Đường dẫn, trạng thái, DOM/API selectors, gotchas
│   └── multi-surface-journeys.md   # Luồng cần 2 tài khoản hoặc 2 bề mặt (예: POS + Admin / Buyer + Seller)
└── bin/
    ├── verify-harness.ts           # Script điều khiển Playwright + CDP (Thu thập Trace, Console, Screenshot)
    ├── doctor.sh                   # Kiểm tra sức khỏe instance (Port, Build version, DB migration xong chưa)
    └── check-feature-map.sh        # Script chạy trên CI kiểm tra Feature Map không bị lệch với Router
```

Giải thích 6 mục bắt buộc trong `SKILL.md`:

1. **`Launch` (Khởi chạy cô lập):** Lệnh chính xác để bật ứng dụng từ một thư mục dữ liệu trống (empty data dir), chạy đủ DB migration và nạp dữ liệu mẫu (seed data). **Quy tắc sống còn:** *"Ready nghĩa là sẵn sàng sử dụng được, không phải chỉ là tiến trình vừa mở cổng"* (một cổng DB có thể trả lời ping trong khi script khởi tạo bảng vẫn đang chạy). Ngoài ra phải hỗ trợ chạy song song nhiều instance (khác cổng, khác profile/schema) để 2 Agent không dẫm chân lên phiên của nhau.

2. **`Doctor` (Khám sức khỏe trước khi lái):** Một lệnh kiểm tra chỉ đọc (read-only check) trả lời câu hỏi *"Instance này có đáng tin để điều khiển không?"* (tiến trình đang sống, đúng phiên bản build vừa biên dịch, port thuộc sở hữu của phiên này, token xác thực còn hạn). Agent bắt buộc chạy `Doctor` trước lượt điều khiển đầu tiên và ngay sau bất kỳ lượt điều khiển nào gặp lỗi bất thường.

3. **`Drive` (Công thức điều khiển thực tế):** Cách dùng Playwright, Chrome DevTools Protocol (CDP), PTY/tmux (cho CLI) hoặc HTTP client với **các selector thật từ repo** (`data-testid`, ARIA labels, route paths), tuyệt đối không bấm theo tọa độ màn hình hay thứ tự phím Tab.

4. **`Evidence` (Tiêu chuẩn bằng chứng):** Quy định rõ bằng chứng phải lưu ở đâu và tiêu chuẩn nghiệm thu:
   - Phải đi qua đúng luồng của người dùng thật (real user path), **cấm gọi các hàm setter nội bộ hay test-only endpoint để làm tắt**.
   - Phải chụp lại **cả hành động lẫn trạng thái kết quả**, đồng thời kiểm chứng cả **tác dụng phụ bên dưới (side effects)**: bản ghi đã thực sự được chèn vào PostgreSQL chưa, message đã vào hàng đợi chưa, file đã ghi ra đĩa chưa.

5. **`Cleanup` (Dọn dẹp an toàn):** Cách tắt các tiến trình do lượt chạy tạo ra. **Nghiêm cấm dùng lệnh `killall` hoặc `pkill` theo tên tiến trình** (vì sẽ giết nhầm ứng dụng của lập trình viên hoặc của Agent khác đang chạy song song); chỉ được tắt đúng PID mà phiên đó đã khởi tạo. Quá trình Cleanup chỉ xóa instance và dữ liệu nháp, **bắt buộc giữ lại thư mục chứa bằng chứng (`Evidence`)** để báo cáo.

6. **`Helpers` (Công cụ đóng gói sẵn):** Mọi thao tác lặp lại (chụp màn hình, lấy CPU trace, kiểm tra DB) phải được đóng gói thành script có quyền thực thi (`chmod +x`) và ghi rõ câu lệnh gọi trong `SKILL.md` để Agent không phải tự viết script tạm bợ mỗi lần chạy.

3.2. Thiết kế tệp `Feature Map` chuẩn (`features/<feature-name>.md`)

Tại sao AI Agent thường thất bại khi nhận một báo cáo lỗi mơ hồ kiểu *"Nút thanh toán bấm không ăn khi áp mã giảm giá"*? Vì nó không biết làm sao để đưa ứng dụng tới đúng màn hình đó với trạng thái dữ liệu hợp lệ. Mỗi tệp trong thư mục `features/` phải tuân thủ **4 tiêu đề H2 cố định** để Agent và script CI có thể quét tự động:

```
# Feature: Luồng Thanh toán & Áp dụng Voucher (Order Checkout)

## 1. User POV & Entry Points
- **Mô tả góc nhìn người dùng:** Khách hàng kiểm tra giỏ hàng, chọn phương thức thanh toán, nhập mã giảm giá và xác nhận
- **Route định tuyến:** `/checkout` (Source: `src/app/router.tsx:84`)
- **Điều kiện tiên quyết (Prerequisites):**
  - Tài khoản đã đăng nhập (Seed user: `qa.buyer@example.com` / Role: `CUSTOMER`).
  - Giỏ hàng phải có ít nhất 1 sản phẩm còn tồn kho (`sku_active_01`).
```

```
- **Phím tắt / Điều hướng nhanh:** Từ màn hình bất kỳ, bấm vào `[data-testid="header-cart-btn"]` → chọn `[data-testid="p

## 2. Stable Handles & Selectors (DOM / API / State)
- Ô nhập mã giảm giá: `getByTestId('voucher-input')` (Source: `src/features/checkout/VoucherBox.tsx:42`)
- Nút áp dụng mã: `getByRole('button', { name: 'Áp dụng' })` hoặc `[data-testid="apply-voucher-btn"]`
- Bảng tổng kết tiền: `[data-testid="order-summary-total"]`
- Nút chốt đơn: `[data-testid="submit-order-btn"]`
- API liên quan:
  - `POST /api/v1/checkout/preview` (Tính toán lại giá)
  - `POST /api/v1/orders` (Tạo đơn hàng — yêu cầu header `Idempotency-Key`)

## 3. Driving Recipe (Cách điều khiển chuẩn)
```bash
# Chạy luồng kiểm chứng chuẩn và xuất bằng chứng ra thư mục /tmp/verify-artifacts/checkout
npx tsx .claude/skills/verify-app/bin/verify-harness.ts \
  --route "/checkout" \
  --seed "cart_with_items" \
  --action "apply-voucher:SUMMER50" \
  --trace-cpu \
  --out "/tmp/verify-artifacts/checkout"
```

4. Gotchas & Side-Effect Proofs (Cạm bẫy & Bằng chứng hậu kiểm)

- **Gotcha 1 (`src/features/checkout/CheckoutForm.tsx:118`):** Nút `submit-order-btn` bị vô hiệu hóa (`disabled`) trong khi API `/api/v1/checkout/preview` đang chạy debounce 300ms. Đừng bấm chốt đơn ngay lập tức sau khi gõ phím; phải đợi trạng thái `[data-busy="false"]`.
- **Bằng chứng bắt buộc (Proof Requirements):**
  1. Ảnh chụp màn hình trước và sau khi áp mã giảm giá (`before.png`, `after.png`).
  2. File `network-log.json` xác nhận `POST /api/v1/orders` trả về HTTP `201 Created`.
  3. Truy vấn DB kiểm chứng bản ghi trong bảng `orders` có `status = 'CONFIRMED'` và bảng `outbox_events` có sự kiện `ORDER_CREATED`.

Đặc biệt, với các luồng nghiệp vụ phức tạp cần 2 tác nhân tương tác (ví dụ: Nhân viên tạo yêu cầu và Sếp phê duyệt, hoặc

---

3.3. Code mẫu: Playwright + CDP Verification Harness (`bin/verify-harness.ts`)

Thay vì để Agent mỗi lần lại tự viết một đoạn script Playwright rời rạc rồi xóa đi, bạn cung cấp sẵn một CLI Harness cho

```typescript
#!/usr/bin/env npx tsx
// .claude/skills/verify-app/bin/verify-harness.ts
import { chromium } from 'playwright';
import * as fs from 'node:fs';
import * as path from 'node:path';

interface VerifyOptions {
  baseUrl: string;
  route: string;
  outDir: string;
  captureCpuTrace: boolean;
}

async function runVerification(opts: VerifyOptions) {
  fs.mkdirSync(opts.outDir, { recursive: true });

  const browser = await chromium.launch({ headless: true });
  const context = await browser.newContext({
    viewport: { width: 1440, height: 900 },
    recordVideo: { dir: opts.outDir }
  });
  const page = await context.newPage();

  // Kết nối trực tiếp vào Chrome DevTools Protocol (CDP) để đo hiệu năng và lấy CPU Trace
  const cdpSession = await context.newCDPSession(page);
  await cdpSession.send('Performance.enable');

  const consoleErrors: string[] = [];
  const networkFailures: Array<{ url: string; status: number }> = [];
```
```

```
  page.on('console', (msg) => {
    if (msg.type() === 'error') consoleErrors.push(msg.text());
  });
  page.on('pageerror', (err) => {
    consoleErrors.push(`UNCAUGHT_PAGE_ERROR: ${err.stack || err.message}`);
  });
  page.on('response', (res) => {
    if (res.status() >= 400) {
      networkFailures.push({ url: res.url(), status: res.status() });
    }
  });

  if (opts.captureCpuTrace) {
    await cdpSession.send('Profiler.enable');
    await cdpSession.send('Profiler.start');
  }

  // Điều hướng tới route cần kiểm chứng
  const targetUrl = new URL(opts.route, opts.baseUrl).toString();
  await page.goto(targetUrl, { waitUntil: 'networkidle' });
  await page.screenshot({ path: path.join(opts.outDir, '01-loaded-state.png'), fullPage: true });

  // Thu thập chỉ số hiệu năng thực tế từ CDP (Layout count, RecalcStyle, JS Heap, Long Tasks)
  const perfMetrics = await cdpSession.send('Performance.getMetrics');

  if (opts.captureCpuTrace) {
    const { profile } = await cdpSession.send('Profiler.stop');
    fs.writeFileSync(
      path.join(opts.outDir, 'cpu-profile.cpuprofile'),
      JSON.stringify(profile, null, 2)
    );
  }

  // Xuất báo cáo tổng hợp dạng JSON để Agent đọc mà không làm tràn Context Window
  const summary = {
    timestamp: new Date().toISOString(),
    targetUrl,
    passed: consoleErrors.length === 0 && networkFailures.length === 0,
    consoleErrors,
    networkFailures,
    cdpMetrics: Object.fromEntries(perfMetrics.metrics.map((m) => [m.name, m.value]))
  };

  fs.writeFileSync(
    path.join(opts.outDir, 'verification-evidence.json'),
    JSON.stringify(summary, null, 2)
  );

  await context.close();
  await browser.close();

  if (!summary.passed) {
    console.error('VERIFICATION_FAILED: Phát hiện lỗi Console hoặc Network. Xem chi tiết tại verification-evidence.json'
    process.exit(1);
  }
  console.log(`VERIFICATION_PASSED: Bằng chứng đã lưu tại ${opts.outDir}`);
}
```

3.4. Chống "Trôi Bản đồ" bằng Cổng kiểm tra CI (`bin/check-feature-map.sh`) & `/maintain-verification-skill`

Lauren chỉ ra rằng một `Feature Map` chỉ có giá trị khi nó **đúng với thực tế** ("A map is only useful while it is true"). Nếu lập trình viên hoặc Agent thêm một màn hình/route mới hoặc đổi tên selector mà quên cập nhật `Feature Map`, Agent sau sẽ bị mù đường.

Vì vậy, cô kết hợp **2 cơ chế bảo vệ**:

1. **Cổng kiểm tra tĩnh trên CI (`bin/check-feature-map.sh`):** Chạy mỗi khi file định tuyến (Router/Controller) thay đổi:
    - Quét toàn bộ các route có thể truy cập trong mã nguồn. Mỗi route bắt buộc phải nằm trong một tệp `features/*.md` hoặc phải được liệt kê rõ trong mục `Unmapped routes` của `features/README.md` kèm lý do không rỗng.
    - Kiểm tra từng tệp `features/*.md` xem có giữ đủ 4 tiêu đề H2 chuẩn hay không.
    - **Fail-closed:** Nếu không đọc được file router hoặc danh sách route rỗng, script trả về mã lỗi `exit 2` làm đỏ CI ngay lập tức.

2. **Quy trình bảo trì định kỳ (`/maintain-verification-skill`):**
    - **Pha 1 – Source Wave (Đọc mã nguồn song song):** Khởi chạy đồng thời các Sub-agent chỉ đọc (read-only), mỗi Sub-agent phụ trách 1 tệp `features/*.md`, đối chiếu tài liệu với code thực tế để phát hiện sự sai lệch (doc drift).
    - **Pha 2 – Live Pass (Lái thử thực tế toàn bộ tính năng):** Ngay cả khi đọc code thấy có vẻ khớp, Coordinator Agent vẫn **bắt buộc bật ứng dụng lên và chạy thử từng tính năng ít nhất 1 lần**.
    - **Pha 3 – Phân loại 3 nhóm lỗi (Triage):**
        - *Doc drift* (Mô tả hoặc selector trong bản đồ bị cũ): Sửa lại tệp `features/*.md`.
        - *Harness gap* (Tính năng vẫn chạy nhưng script điều khiển chưa bấm tới được): Nâng cấp script điều khiển và chạy lại để chứng minh.
        - *Product gap* (Chính tính năng của sản phẩm bị hỏng thật): Ghi nhận lại thành báo cáo lỗi cho con người, **không tự ý sửa code sản phẩm chung vào PR bảo trì bản đồ** để giữ tính nguyên tử của PR!

PHẦN 4: Vận hành Tự hành Qua Đêm (`show-me-your-work`), Điều phối Vòng Ngoài & Ranh giới Rủi ro

Khi Vòng Trong (Inner Loop) đã khép kín nhờ `pstack`, Tháp Kiểm soát CI và `Feature Map`, làm thế nào để vận hành hàng chục Agent chạy qua đêm và xử lý hàng nghìn PR mà vẫn kiểm soát được chất lượng?

4.1. Giao thức Kiểm toán Tự hành: `/show-me-your-work` và Nhật ký Quyết định (`decision-log.tsv`)

Khi bạn giao việc cho Agent chạy tự động nhiều giờ liền qua lệnh `/loop` (ví dụ: *"Tôi đi ngủ đây, hãy di trú toàn bộ các nơi gọi sang Parser mới trong một worktree riêng, xóa API cũ và giữ nhật ký quyết định"*), rủi ro lớn nhất không phải là Agent dừng lại, mà là **Agent gặp ngõ cụt rồi tự ý thỏa hiệp, hạ thấp tiêu chuẩn hoặc lách luật để báo cáo hoàn thành**.

Trong `pstack`, skill `/show-me-your-work` ép Agent phải duy trì một tệp nhật ký quyết định dạng bảng phân cách bằng dấu Tab (**append-only TSV**):

```
step_id     timestamp    phase  decision_made      alternatives_rejected     principles_cited    verification_evidence
01    2026-10-20T23:10:00Z      scaffold     Tạo Branded Type OrderId tại ranh giới parser Dùng string thuần và kiểm tra
02    2026-10-20T23:42:15Z      migrate      Chuyển 14 callers trong module billing sang parseOrder()   Giữ hàm parseLeg
03    2026-10-21T00:15:30Z      verify Chạy E2E Checkout flow sau khi xóa toàn bộ API cũ   Chỉ chạy unit test của parser
```

- **Kiểm toán chéo trước khi bàn giao (Cross-model Audit):** Trước khi kết thúc phiên chạy đêm và báo cáo lại cho con người vào buổi sáng, một mô hình khác họ (`judge` / `peer`) sẽ đọc lại toàn bộ tệp `decision-log.tsv` này, đối chiếu từng dòng với lịch sử `git diff` thực tế để đảm bảo Agent thực thi không khai khống bằng chứng.
- Khi thức dậy, thay vì phải đọc hàng nghìn dòng code, kỹ sư chỉ cần đọc bảng TSV này trong 2 phút để nắm toàn bộ các quyết định kiến trúc mà Agent đã thực hiện trong đêm.

4.2. Phân loại Tác vụ: Khi nào được "Auto-Merge" và Khi nào phải "Chốt chặn Cứng"?

Để áp dụng thành công vào các hệ thống doanh nghiệp (như Ngân hàng, Fintech, Thương mại điện tử, POS hay Hệ thống Lõi), Kiến trúc sư bắt buộc phải phân tách rạch ròi giữa **Quyết định Cửa hai chiều (Reversible)** và **Quyết định Cửa một chiều (Irreversible)** theo đúng nguyên tắc `principle-never-block-on-the-human`:

Cấp độ Tự chủ	Loại thay đổi trong Hệ thống	Chế độ vận hành với AI Agent	Cơ chế Kiểm chứng Bắt buộc
Cấp độ 1: Auto-Merge + Post-Merge Sampling *(Tự động merge, con người kiểm tra lấy mẫu sau)*	- Sửa lỗi UI/UX, CSS, hiển thị. - Tối ưu hiệu năng cục bộ (Render, Virtualization). - Refactoring nội bộ trong 1 module có độ phủ test cao. - Cập nhật `Feature Map`, viết thêm Test, dọn dẹp Dead Code.	Cho phép chạy Playbook `Autopilot-full` hoặc `Shipping` để Agent tự động merge khi qua hết các cổng CI và Independent Reviewer Agent.	- Hard Linters + Dependency Graph. - Playwright/CDP Live Verification. - Tách nhỏ thành **Atomic PRs** (50–300 LOC) để có thể `git revert` trong 5 giây nếu phát hiện bất thường.
Cấp độ 2: Autopilot-Stack + Human Stamp *(Agent làm*	- Tính năng nghiệp vụ mới xuyên qua nhiều module.	Chạy Playbook `Autopilot-stack`: Agent tự thiết kế (`/architect`), tự code, tự chạy `/interrogate` đa mô	- Chạy `/blast-radius` chứng minh các dịch vụ tiêu thụ (consumers) không bị vỡ.

Cấp độ Tự chủ	Loại thay đổi trong Hệ thống	Chế độ vận hành với AI Agent	Cơ chế Kiểm chứng Bắt buộc
trọn gói chuỗi PR, con người duyệt nhát cuối)	- Thay đổi hợp đồng giao tiếp nội bộ (Internal API, Kafka/RabbitMQ Event Payload). - Thay đổi cấu hình Caching (Redis TTL/Invalidation).	hình, tự xếp chồng chuỗi PR xanh toàn bộ, nhưng **dừng lại ở bước bấm nút Merge cuối cùng** để Architect duyệt.	- Chạy `multi-surface-journeys` kiểm chứng đầu-cuối giữa Producer và Consumer.
Cấp độ 3: Strict Human-in-the-Loop + Formal Verification *(Cửa một chiều — Nghiêm cấm Auto-Merge)*	- **Database Schema Migration** (xóa cột, đổi kiểu dữ liệu bảng lớn). - **Logic hạch toán tài chính, đối soát, thanh toán.** - **Cơ chế khóa phân tán (Distributed Locks), State Machine đa luồng, Bảo mật & Phân quyền (AuthZ).**	Bắt buộc khóa quyền push/merge tự động. Sử dụng kỹ năng mới nhất trong `pstack`: **`/create-formal-verification-skill`** và **`/maintain-formal-verification-skill`**.	Khi bài test thông thường không thể vét cạn các trường hợp tranh chấp đồng thời (race conditions, retry storms), dùng **Formal Verification** (kiểm chứng hình thức bằng **TLA+ model checking**, **Lean proofs**, hoặc **Property-Based Testing / Jepsen-style fault injection**) để chứng minh tính bất biến (invariants) trước khi con người ký duyệt.

4.3. Lộ trình 5 Bước Triển khai Thực tế cho Đội ngũ Kỹ thuật

Nếu bạn muốn đưa mô hình của Lauren Tan vào dự án hiện tại mà không gây sốc cho hệ thống hay đốt token vô ích, hãy thực hiện tuần tự theo 5 giai đoạn sau:

1. **Thiết lập Bộ khung Điều phối (pstack) và Phân vai Mô hình**

   Tuần 1 — Chuẩn hóa quy trình cho 1 Agent đơn lẻ

   Cài đặt `pstack` (trên Cursor qua `/add-plugin pstack` hoặc trên Claude Code / OpenCode / Pi qua `pstack-claude`). Chạy `/setup-pstack` để tách bạch mô hình viết code nhanh (`worker`/`builder`) và mô hình suy luận sâu làm giám khảo (`judge`/`peer`). Bắt đầu mọi tác vụ phức tạp bằng `/poteto-mode` kết hợp với `/how` và `/why` để chấm dứt tình trạng Agent đoán mò nguyên nhân lỗi.

2. **Xây dựng Verification Skill và Feature Map cho 3–5 Luồng cốt lõi**

   Tuần 2 — Gỡ bỏ con người khỏi vai trò 'Người chạy thử hộ AI'

   Chạy `/create-verification-skill` để khảo sát codebase và tạo bộ khung `.claude/skills/verify-<app>/` (hoặc `.cursor/skills/verify-<app>/`). Viết script `Launch`, `Doctor` và đóng gói Playwright/CDP (cho Web/Electron) hoặc Testcontainers/HTTP Harness (cho Backend API). Khởi tạo `Feature Map` cho 3–5 luồng nghiệp vụ quan trọng nhất và gắn `bin/check-feature-map.sh` vào CI.

3. **Nâng cấp Tháp Kiểm soát Cứng (Hard Constraints) bằng /correct**

   Tuần 3 — Biến nhận xét Code Review thành Lỗi Biên dịch / Linter

   Rà soát lại 20–30 PR gần nhất mà bạn phải sửa tay hoặc nhắc nhở AI. Chạy `/correct` để phân loại các lỗi lặp lại và mã hóa chúng thành **ArchUnit tests** (đối với Java/Spring Boot) hoặc **Dependency Cruiser + Custom ESLint rules** (đối với TypeScript). Đảm bảo mọi thông báo lỗi của Linter/ArchUnit đều chứa sẵn câu hướng dẫn *"Cách sửa: ..."* để Agent tự phục hồi.

4. **Mở rộng Song song hóa Cô lập (Worktrees & Cloud Agents)**

   Tuần 4 — Từ 1 Agent lên 5–10 Agents chạy đồng thời

   Áp dụng nguyên tắc `principle-separate-before-serializing-shared-state`: Mỗi tác vụ hoặc mỗi nhánh thử nghiệm (`/arena`) phải chạy trong một **Git Worktree riêng biệt** hoặc một container/VM độc lập để các Agent không ghi đè file hay làm hỏng trạng thái Database của nhau. Sử dụng `/show-me-your-work` để kiểm toán các phiên chạy dài qua bảng `decision-log.tsv`.

5. **Tự động hóa Vòng Ngoài (Triage Bot & Cá nhân hóa bằng /automate-me)**

   Giai đoạn Trưởng thành — Vận hành như một Bếp trưởng

   Thiết lập các Agent vòng ngoài (tương tự bot **Benny**) lắng nghe sự kiện từ Slack/Linear/Sentry: khi có báo cáo lỗi mới, Agent tự đọc `Feature Map`, tự bật môi trường cô lập tái hiện lỗi và gắn nhãn xem lỗi đã được sửa trên `main` hay chưa trước khi con người chạm vào. Cuối cùng, chạy `/automate-me` để trích xuất khẩu vị kiến trúc riêng của bạn từ lịch sử làm việc thành một chế độ điều phối mang dấu ấn cá nhân.

Bạn cũng có thể theo dõi thêm Buổi trò chuyện trực tiếp giữa Matt Pocock và Lauren Tan (Poteto) để xem trực tiếp cách cô vận hành `pstack` và giải đáp các câu hỏi thực tế về việc duy trì chất lượng kiểm chứng khi xuất bản hàng nghìn PR mỗi tháng.

http://googleusercontent.com/youtube_content/1

User prompt: Phân tích, nghiên cứu, tìm hiểu chi tiết tất cả triển khai chi tiết cấu phần vào hệ thống thực tế Ví dụ: Thiết kế bộ SKILL.md và Feature Map mẫu cho dự án Xây dựng trọn bộ ArchUnit & ESLint chống AI Slop Thiết lập quy trình Tmux + Git Worktree chạy đa Agent ... ...

Response: Để đưa triết lý của Lauren Tan (`@poteto`) vào vận hành thực tế ở cấp độ doanh nghiệp (Enterprise Production), bạn không thể chỉ dừng lại ở việc cài một plugin rồi gõ lệnh, mà phải xây dựng một **Hệ thống Khung giá đỡ Kỹ thuật (Engineering Harness)** đồng bộ từ tầng mã nguồn, trình biên dịch, công cụ kiểm thử tự động cho đến hạ tầng điều phối đa tác nhân.

Dưới đây là bản thiết kế kỹ thuật chi tiết và mã nguồn triển khai thực tế cho **5 cấu phần cốt lõi**, được tối ưu hóa cho kiến trúc hệ thống hiện đại (Backend **Java 25 + Spring Boot + PostgreSQL CQRS + Kafka/RabbitMQ + Redis** và Frontend/Automation **TypeScript + React + Playwright + Vitest** trên môi trường **Linux/Tmux/Git Worktrees**).

MODULE 1: Thiết kế Trọn bộ `Verification Skill` (Chuẩn 6 Phần) & `Feature Map` Tự động

Trong `pstack`, kỹ năng kiểm chứng (`verify-[app]`) là điều kiện tiên quyết để AI thoát khỏi cảnh "đoán mò" và giải phóng con người khỏi vai trò chạy thử nghiệm thủ công. Một Verification Skill đạt chuẩn của Lauren Tan phải tuân thủ nguyên tắc **"Khảo sát kho mã nguồn, không phỏng vấn con người" (Interview the repo, not the user)** và tuyệt đối không chứa dữ liệu giả (placeholders).

1.1. Cấu trúc thư mục chuẩn trong Repository

```
.claude/skills/verify-platform/          # (Hoặc .cursor/skills/verify-platform/)
├── SKILL.md                             # Tài liệu điều khiển chuẩn 6 phần
├── features/                            # FEATURE MAP: Bản đồ định vị cho Agent
│   ├── README.md                        # Chỉ mục tính năng + Danh sách Unmapped routes
│   ├── order-checkout-flow.md           # Bản đồ chi tiết luồng Thanh toán
│   └── multi-surface-journeys.md        # Luồng liên thông (POS/Client -> Kafka -> Admin)
└── bin/
    ├── launch-isolated.sh               # Khởi chạy ứng dụng theo Slot cô lập
    ├── doctor.sh                        # Khám sức khỏe instance trước và sau khi chạy
    ├── verify-harness.ts                # Công cụ lái Playwright + CDP + Kiểm tra DB/Kafka
    ├── cleanup-slot.sh                  # Tắt đúng PID của phiên, bảo toàn thư mục Evidence
    └── check-feature-map.sh             # Cổng CI chặn đứng nếu Feature Map lệch với Code
```

1.2. Nội dung chi tiết file `.claude/skills/verify-platform/SKILL.md`

````
---
name: verify-platform
description: Khởi chạy cô lập, điều khiển tự động và thu thập bằng chứng runtime (UI, CDP Trace, API, PostgreSQL, Kafka)
disable-model-invocation: false
---

# Quy trình Kiểm chứng Thực tế (Platform Verification Skill)

Không bao giờ báo cáo hoàn thành công việc chỉ vì code biên dịch thành công (`BUILD SUCCESSFUL`) hoặc Unit Test báo xanh

## 1. Launch (Khởi chạy Cô lập theo Slot)

Mỗi Agent bắt buộc chạy trên một `AGENT_SLOT` riêng biệt (từ `01` đến `20`) để không xung đột cổng mạng và dữ liệu với l

```bash
# Khởi tạo môi trường cô lập cho Slot 01 (Backend: 18081, UI: 13001, CDP: 19221, DB Schema: slot_01)
export AGENT_SLOT="01"
./.claude/skills/verify-platform/bin/launch-isolated.sh --slot "$AGENT_SLOT" --seed "standard_qa"
````

- **Định nghĩa Ready:** Tiến trình mở cổng chưa được tính là Ready. Script `launch-isolated.sh` chỉ trả về mã `0` khi:
    1. Flyway/Liquibase hoàn tất migration trên schema `slot_01`.
    2. Dữ liệu mẫu (`standard_qa`) đã nạp xong.
    3. Endpoint `/actuator/health/readiness` trả về `{"status":"UP"}` và trang chủ UI trả về HTTP `200`.

2. Doctor (Khám sức khỏe Instance)

Bắt buộc chạy lệnh `doctor.sh` **trước lượt điều khiển đầu tiên** và **ngay sau bất kỳ lượt điều khiển nào gặp lỗi bất thường** để đảm bảo bạn không kiểm thử nhầm vào bản build cũ hoặc tiến trình bị treo:

```
./.claude/skills/verify-platform/bin/doctor.sh --slot "$AGENT_SLOT"
```
```

Lệnh này kiểm tra chỉ đọc (read-only) 4 điều kiện:

1. PID trong `/tmp/agent-slot-${AGENT_SLOT}/pids.env` còn sống và sở hữu đúng cổng đã cấp.
2. Header `X-Build-Git-Sha` trả về từ `/actuator/info` khớp với `git rev-parse HEAD` hoặc bản build vừa biên dịch.
3. Kết nối từ ứng dụng tới PostgreSQL schema `slot_${AGENT_SLOT}` và Redis DB `${AGENT_SLOT}` thông suốt.
4. Token xác thực thử nghiệm trong `/tmp/agent-slot-${AGENT_SLOT}/auth-state.json` còn hiệu lực trên 15 phút.

3. Drive (Điều khiển Thực tế)

Trước khi điều khiển bất kỳ tính năng nào, **bắt buộc đọc tệp tương ứng trong `./features/`** để lấy đúng Route, trạng thái tiên quyết và bộ chọn (`data-testid`).

- **Nghiêm cấm:** Không click theo tọa độ `(x, y)`, không đoán mò selector khi chưa đọc `Feature Map`, không gọi API nội bộ chỉ dành cho test để bỏ qua các bước trên giao diện.
- **Công cụ điều khiển chuẩn:** Sử dụng `verify-harness.ts` để thực thi kịch bản tương tác kết hợp thu thập Chrome DevTools Protocol (CDP) trace:

```
npx tsx ./.claude/skills/verify-platform/bin/verify-harness.ts \
  --slot "$AGENT_SLOT" \
  --feature "order-checkout-flow" \
  --scenario "apply-voucher-and-pay" \
  --out "/tmp/verify-evidence/slot-${AGENT_SLOT}/$(date +%Y%m%d-%H%M%S)"
```

4. Evidence (Tiêu chuẩn Nghiệm thu Bằng chứng)

Một lượt kiểm chứng chỉ được công nhận là **PASS** khi thư mục đầu ra (`--out`) chứa đầy đủ 4 loại bằng chứng sau và tất cả đều hợp lệ:

1. **Bằng chứng Thị giác (`01-before.png`, `02-after.png`):** Ảnh chụp màn hình trước và sau khi thực hiện hành động trên giao diện thật.
2. **Bằng chứng Sức khỏe Runtime (`runtime-report.json`):** Mảng `consoleErrors` và `uncaughtExceptions` phải rỗng (`[]`); không có request API nào trả về HTTP 5xx hoặc 4xx ngoài dự kiến.
3. **Bằng chứng Hiệu năng (`cdp-metrics.json` / `cpu-trace.cpuprofile`):** Với các tác vụ UI/Perf, không có frame nào trong thao tác chính gây Long Task vượt ngưỡng ngân sách được quy định.
4. **Bằng chứng Tác dụng phụ Hạ tầng (`side-effects.json`):** Xác nhận trạng thái thực tế trong PostgreSQL (bản ghi đã chuyển đúng trạng thái) và Outbox/Kafka (sự kiện nghiệp vụ đã được phát đi).

5. Cleanup (Dọn dẹp An toàn)

Sau khi thu thập xong bằng chứng, giải phóng tài nguyên của Slot:

```
./.claude/skills/verify-platform/bin/cleanup-slot.sh --slot "$AGENT_SLOT"
```

- **Quy tắc an toàn tuyệt đối:** Tuyệt đối **CẤM** chạy `killall java`, `pkill -f node` hoặc `docker compose down` vì sẽ giết chết tiến trình của lập trình viên hoặc các Agent ở Slot khác. Script `cleanup-slot.sh` chỉ gửi tín hiệu `SIGTERM` tới đúng các PID được ghi trong `/tmp/agent-slot-${AGENT_SLOT}/pids.env`, xóa schema nháp nhưng **giữ nguyên thư mục `/tmp/verify-evidence/...`** để đính kèm vào báo cáo PR.

6. Helpers (Các công cụ hỗ trợ đóng gói sẵn)

Tuyệt đối không tự viết các đoạn script bash/python dùng một lần để truy vấn DB hay đọc trace. Hãy dùng các công cụ đã được kiểm thử sẵn tại `./bin/`:

- `./bin/query-slot-db.sh --slot "$AGENT_SLOT" --sql "SELECT status, total_amount FROM orders WHERE id = '...'"`: Truy vấn trực tiếp vào PostgreSQL schema của Slot hiện tại.
- `./bin/inspect-kafka-outbox.sh --slot "$AGENT_SLOT" --aggregate-id "..."`: Kiểm tra sự kiện đã được ghi vào bảng `outbox_events` và đẩy lên Kafka topic của Slot hay chưa.
- `./bin/check-feature-map.sh`: Kiểm tra tính đồng bộ giữa Router trong code và thư mục `./features/`.

1.3. Mẫu `Feature Map` Chuẩn (`features/README.md` & `features/order-checkout-flow.md`)

A. File chỉ mục `features/README.md` (Quản lý độ phủ bản đồ)

```markdown
# Chỉ mục Bản đồ Tính năng (Feature Map Index)
```

```
Mọi route trong hệ thống bắt buộc phải được ánh xạ vào một tệp bản đồ dưới đây hoặc nằm trong danh sách miễn trừ có lý d

## 1. Các tính năng đã ánh xạ (Mapped Features)
- [Luồng Thanh toán & Đặt hàng](./order-checkout-flow.md) — Routes: `/cart`, `/checkout`, `/orders/:orderId/confirmation
- [Hành trình Đa bề mặt: Đặt hàng -> Kho vận -> Đối soát](./multi-surface-journeys.md) — Routes: `/checkout` (Client) +

## 2. Các Route miễn trừ chưa ánh xạ (Unmapped Routes — Bắt buộc nêu lý do)
- `/healthz`, `/actuator/**` — Lý do: Endpoint kỹ thuật dành cho hạ tầng giám sát và script `doctor.sh`, không có giao d
- `/dev/playground` — Lý do: Môi trường thử nghiệm component nội bộ chỉ bật khi `NODE_ENV=development`, không thuộc luồn
```

B. File bản đồ chi tiết `features/order-checkout-flow.md` (Đúng 4 tiêu đề H2 bắt buộc)

```
# Feature: Luồng Thanh toán & Đặt hàng (Order Checkout Flow)

## 1. User POV & Entry Points
- **Góc nhìn người dùng:** Khách hàng rà soát giỏ hàng, chọn địa chỉ nhận hàng, áp dụng mã khuyến mãi và xác nhận thanh
- **Routes định tuyến:**
  - `/cart` (`web-app/src/routes/AppRouter.tsx:45`)
  - `/checkout` (`web-app/src/routes/AppRouter.tsx:52`)
  - `/orders/:orderId/confirmation` (`web-app/src/routes/AppRouter.tsx:59`)
- **Điều kiện tiên quyết (Prerequisites):**
  - Đã nạp trạng thái đăng nhập từ `/tmp/agent-slot-${AGENT_SLOT}/auth-state.json` (User: `customer.verified@example.com
  - Giỏ hàng đã được seed qua kịch bản `standard_qa` (chứa tối thiểu 1 SKU có `available_stock >= 5`).

## 2. Stable Handles & Selectors
- **Giao diện (DOM Selectors):**
  - Nút chuyển sang thanh toán từ giỏ hàng: `[data-testid="cart-proceed-checkout-btn"]` (`CartSummary.tsx:78`)
  - Ô nhập mã giảm giá: `[data-testid="checkout-voucher-input"]` (`VoucherSection.tsx:31`)
  - Nút áp dụng mã: `[data-testid="checkout-voucher-apply-btn"]` (`VoucherSection.tsx:44`)
  - Nhãn hiển thị số tiền giảm: `[data-testid="checkout-discount-amount"]` (`OrderTotals.tsx:62`)
  - Nút xác nhận thanh toán: `[data-testid="checkout-submit-order-btn"]` (`CheckoutFooter.tsx:95`)
  - Mã đơn hàng trên màn hình thành công: `[data-testid="confirmation-order-code"]` (`OrderConfirmationPage.tsx:29`)
- **Backend API & Messaging Contracts:**
  - `POST /api/v1/vouchers/validate` -> `VoucherController.java:41`
  - `POST /api/v1/orders` (Yêu cầu header `Idempotency-Key`) -> `OrderCommandController.java:58`
  - Sự kiện Outbox phát sinh: `topic = slot_${AGENT_SLOT}.orders.events`, `event_type = OrderPlacedEvent`

## 3. Driving Recipe
```bash
# Chạy kiểm chứng tự động toàn trình cho luồng áp mã giảm giá và đặt hàng
npx tsx ./.claude/skills/verify-platform/bin/verify-harness.ts \
  --slot "$AGENT_SLOT" \
  --route "/checkout" \
  --voucher "WELCOME50K" \
  --expect-discount "50000" \
  --out "/tmp/verify-evidence/slot-${AGENT_SLOT}/checkout-run"
```

4. Gotchas & Side-Effect Proofs

- **Gotcha 1 (Debounce & Optimistic Lock):** Khi điền mã vào `[data-testid="checkout-voucher-input"]` và bấm nút áp dụng, nút `[data-testid="checkout-submit-order-btn"]` sẽ chuyển sang trạng thái `aria-busy="true"` trong khoảng 150ms–400ms để tính lại thuế và phí vận chuyển. **Bắt buộc đợi selector `[data-testid="checkout-submit-order-btn"][aria-busy="false"]` trước khi click đặt hàng.**
- **Gotcha 2 (Idempotency Key):** Nếu thử nghiệm hành vi retry khi mạng chập chờn, phải giữ nguyên cùng một giá trị UUID cho header `Idempotency-Key` để Backend trả về kết quả đã xử lý thay vì tạo 2 đơn hàng trùng lặp.
- **Bằng chứng Hậu kiểm Bắt buộc (Side-Effect Proofs):**
  1. Truy vấn PostgreSQL: `./bin/query-slot-db.sh --slot "$AGENT_SLOT" --sql "SELECT status, discount_amount FROM orders WHERE order_code = '<code_trên_UI>'"` phải trả về `status = 'PAID'` và `discount_amount = 50000`.
  2. Truy vấn Outbox: `./bin/inspect-kafka-outbox.sh --slot "$AGENT_SLOT" --aggregate-id "<order_id>"` phải trả về bản ghi có `published = true`.

```
---

### 1.4. Script Cổng CI Chống lệch Bản đồ (`bin/check-feature-map.sh`)

```bash
#!/usr/bin/env bash
# .claude/skills/verify-platform/bin/check-feature-map.sh
# Cổng CI Fail-Closed: Đảm bảo mọi Route trong code đều có mặt trong Feature Map và giữ đúng 4 mục H2 chuẩn.
set -euo pipefail
```

```
ROUTER_FILE="web-app/src/routes/AppRouter.tsx"
FEATURES_DIR=".claude/skills/verify-platform/features"
INDEX_FILE="${FEATURES_DIR}/README.md"

if [[ ! -f "$ROUTER_FILE" ]]; then
  echo "[FAIL-CLOSED] Không tìm thấy file định tuyến tại: $ROUTER_FILE" >&2
  exit 2
fi

# 1. Trích xuất toàn bộ đường dẫn route từ mã nguồn
mapfile -t CODE_ROUTES < <(grep -oE 'path:\s*["'\'']([^"'\'']*)["'\'']' "$ROUTER_FILE" | sed -E "s/path:\s*['\"]([^'\"]

if [[ ${#CODE_ROUTES[@]} -eq 0 ]]; then
  echo "[FAIL-CLOSED] Không trích xuất được bất kỳ route nào từ $ROUTER_FILE. Kiểm tra lại cú pháp parser!" >&2
  exit 2
fi

# 2. Kiểm tra từng route trong code xem đã được ghi nhận trong features/*.md hoặc mục Unmapped trong README.md chưa
MISSING_ROUTES=()
for route in "${CODE_ROUTES[@]}"; do
  if ! grep -rqF "$route" "$FEATURES_DIR"; then
    MISSING_ROUTES+=("$route")
  fi
done

if [[ ${#MISSING_ROUTES[@]} -gt 0 ]]; then
  echo "❌ [FEATURE-MAP DRIFT DETECTED] Phát hiện các route mới trong code chưa được ánh xạ vào Feature Map:" >&2
  for mr in "${MISSING_ROUTES[@]}"; do
    echo "   - $mr" >&2
  done
  echo "👉 CÁCH SỬA: Hãy cập nhật file tương ứng trong ${FEATURES_DIR}/*.md hoặc chạy skill /maintain-verification-skil
  exit 1
fi

# 3. Kiểm tra cấu trúc 4 tiêu đề H2 bắt buộc trên mọi tệp bản đồ (trừ README.md)
REQUIRED_SECTIONS=(
  "## 1. User POV & Entry Points"
  "## 2. Stable Handles & Selectors"
  "## 3. Driving Recipe"
  "## 4. Gotchas & Side-Effect Proofs"
)

for md_file in "${FEATURES_DIR}"/*.md; do
  [[ "$(basename "$md_file")" == "README.md" ]] && continue
  for section in "${REQUIRED_SECTIONS[@]}"; do
    if ! grep -qF "$section" "$md_file"; then
      echo "❌ [SCHEMA VIOLATION] File $md_file thiếu mục bắt buộc: '$section'" >&2
      echo "👉 CÁCH SỬA: Giữ đúng 4 tiêu đề H2 chuẩn của Feature Map để Agent định vị chính xác." >&2
      exit 1
    fi
  done
done

echo "✅ [FEATURE-MAP OK] Toàn bộ ${#CODE_ROUTES[@]} routes và cấu trúc bản đồ đều đồng bộ với mã nguồn."
```

MODULE 2: Xây dựng Trọn bộ "Hard Constraints" (ArchUnit & ESLint) Chống AI Slop

Triết lý cốt lõi giúp Lauren Tan không bị sa lầy vào việc ngồi đọc và bình luận từng dòng trên hàng nghìn PR là nguyên tắc **`principle-encode-lessons-in-structure`**:

> *"Khi bạn phải nhắc lại cùng một lỗi lần thứ hai, đừng ghi thêm chữ vào file Markdown. Hãy biến nó thành lỗi biên dịch hoặc lỗi Linter làm đỏ CI ngay lập tức."*

Tuy nhiên, một quy tắc Linter hoặc ArchUnit thông thường chỉ báo lỗi chung chung như `Rule 'X' was violated`. Khi AI Agent gặp thông báo cụ thể nhưng thiếu hướng dẫn, nó sẽ tìm cách "lách luật" (ví dụ: thêm `@SuppressWarnings`, đổi tên hàm hoặc tạo wrapper giả). Do đó, **mọi thông báo lỗi trong hệ thống của bạn phải được viết theo cấu trúc Self-Healing Prompt (Thông báo lỗi tự dẫn đường)**:

1. **`[VIOLATION: Tên nguyên tắc]`**: Chỉ rõ quy tắc kiến trúc nào vừa bị phá vỡ.
2. **`[WHY]`**: Giải thích ngắn gọn hậu quả kỹ thuật (ví dụ: treo Connection Pool, tụt FPS, rò rỉ dữ liệu).
3. **`[REMEDIATION]`**: Đưa ra công thức sửa cụ thể (tên class/package/pattern chuẩn cần dùng) để Agent tự sửa đúng ngay ở lượt chạy tiếp theo.

2.1. Bộ Quy tắc Cưỡng chế Kiến trúc Backend (`AgentHardConstraintsArchTest.java` cho Java 25 & Spring Boot)

Đặt tệp này vào module kiểm thử kiến trúc dùng chung của dự án Gradle. Bất kỳ Agent nào viết code phạm vào 7 lỗi kinh điển dưới đây sẽ bị chặn đứng ngay khi chạy `./gradlew check`:

```
package vn.enterprise.architecture;

import com.tngtech.archunit.base.DescribedPredicate;
import com.tngtech.archunit.core.domain.JavaCall;
import com.tngtech.archunit.core.domain.JavaClass;
import com.tngtech.archunit.core.domain.JavaField;
import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchCondition;
import com.tngtech.archunit.lang.ArchRule;
import com.tngtech.archunit.lang.ConditionEvents;
import com.tngtech.archunit.lang.SimpleConditionEvent;
import jakarta.persistence.Entity;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.RestController;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Date;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.*;

@AnalyzeClasses(
    packages = "vn.enterprise.platform",
    importOptions = {ImportOption.DoNotIncludeTests.class}
)
public class AgentHardConstraintsArchTest {

    // ========================================================================
    // LUẬT 1: CƯỠNG CHẾ LOGICAL CQRS (Tách biệt cứng Command/Master và Query/Replica)
    // ========================================================================
    @ArchTest
    static final ArchRule cqrs_query_side_must_never_mutate_state_or_depend_on_command_side =
        noClasses()
            .that().resideInAPackage("..query..")
            .should().dependOnClassesThat().resideInAPackage("..command..")
            .orShould().callMethodWhere(new DescribedPredicate<>("gọi hàm ghi dữ liệu (save/delete/update/flush)") {
                @Override
                public boolean test(JavaCall<?> call) {
                    String name = call.getTarget().getName();
                    return name.startsWith("save") || name.startsWith("delete")
                        || name.startsWith("update") || name.equals("flush");
                }
            })
            .because("[VIOLATION: principle-boundary-discipline & Logical CQRS]\n"
                + "[WHY]: Package '..query..' định tuyến vào PostgreSQL Read-Replica chỉ đọc. Mọi thao tác ghi hoặc phụ
                + "[REMEDIATION]: (1) Nếu chỉ đọc dữ liệu, hãy dùng Projection/ReadRepository trong '..query..'. (2) Nếu

    // ========================================================================
    // LUẬT 2: CHỐNG TREO DB CONNECTION POOL (Cấm gọi Network/Broker/S3 trong @Transactional)
    // ========================================================================
    @ArchTest
    static final ArchRule transactional_methods_must_never_perform_external_network_or_broker_io =
        noClasses()
            .that().areAnnotatedWith(Transactional.class)
            .or().haveOnlyMethodsThat(DescribedPredicate.describe("có @Transactional",
                m -> m.isAnnotatedWith(Transactional.class)))
            .should().dependOnClassesThat().resideInAnyPackage(
                "..infrastructure.client..",
                "..infrastructure.storage.minio..",
                "org.springframework.kafka.core..",
                "org.springframework.amqp.rabbit.core..",
                "org.springframework.web.client..",
```

```
            "io.minio.."
        )
        .because("[VIOLATION: principle-foundational-thinking — Database Connection Starvation]\n"
            + "[WHY]: Gọi HTTP API bên ngoài, MinIO S3 hoặc đẩy message trực tiếp vào Kafka/RabbitMQ bên trong khối
            + "[REMEDIATION]: (1) Với Kafka/RabbitMQ: Ghi bản ghi vào bảng 'outbox_events' thông qua OutboxPublisher

    // ========================================================================
    // LUẬT 3: CÔ LẬP MODULE THEO TÍNH NĂNG (Feature Collocation & Encapsulation)
    // ========================================================================
    @ArchTest
    static final ArchRule module_internal_packages_must_remain_strictly_private =
        classes()
            .that().resideInAPackage("vn.enterprise.platform.modules.(*)..internal..")
            .should().onlyBeAccessed().byClassesThat()
            .resideInAPackage("vn.enterprise.platform.modules.$1..")
            .because("[VIOLATION: principle-minimize-reader-load — Cross-Module Internal Leakage]\n"
                + "[WHY]: AI Agent không được phép import trực tiếp Repository, Entity hay Service nội bộ ('..internal..
                + "[REMEDIATION]: Chỉ được phép giao tiếp giữa các module thông qua Public Facade/Port đặt tại 'vn.enter

    // ========================================================================
    // LUẬT 4: KỶ LUẬT RANH GIỚI API (Cấm lộ JPA Entity ra tầng REST / Messaging)
    // ========================================================================
    @ArchTest
    static final ArchRule rest_controllers_must_never_leak_jpa_entities =
        methods()
            .that().areDeclaredInClassesThat().areAnnotatedWith(RestController.class)
            .should().notHaveRawReturnType(DescribedPredicate.describe("JPA @Entity class",
                c -> c.isAnnotatedWith(Entity.class) || c.getPackageName().contains(".persistence.")))
            .because("[VIOLATION: principle-boundary-discipline — Persistence Model Leakage]\n"
                + "[WHY]: Trả trực tiếp JPA @Entity ra ngoài RestController gây lỗi LazyInitializationException, lộ cấu
                + "[REMEDIATION]: Tạo Java 'record' DTO bất biến (immutable) tại package '..api.dto..' kèm Jakarta Valid

    // ========================================================================
    // LUẬT 5: BẮT BUỘC TÍNH LŨY ĐẲNG (Idempotency Discipline cho Mutation & Consumer)
    // ========================================================================
    @ArchTest
    static final ArchRule state_mutating_endpoints_and_message_consumers_must_enforce_idempotency =
        methods()
            .that().areAnnotatedWith(PostMapping.class)
            .or().areAnnotatedWith(PutMapping.class)
            .or().areAnnotatedWith(PatchMapping.class)
            .or().areAnnotatedWith(KafkaListener.class)
            .or().areAnnotatedWith(RabbitListener.class)
            .should(new ArchCondition<>("khai báo @Idempotent hoặc kiểm tra qua IdempotencyGuard") {
                @Override
                public void check(com.tngtech.archunit.core.domain.JavaMethod method, ConditionEvents events) {
                    boolean hasAnnotation = method.getAnnotations().stream()
                        .anyMatch(a -> a.getRawType().getSimpleName().equals("Idempotent"));
                    boolean callsGuard = method.getMethodCallsFromSelf().stream()
                        .anyMatch(c -> c.getTargetOwner().getSimpleName().contains("Idempotency"));
                    if (!hasAnnotation && !callsGuard) {
                        events.add(SimpleConditionEvent.violated(method,
                            String.format("Method %s không có cơ chế bảo vệ lũy đẳng (Idempotency).", method.getFullName
                    }
                }
            })
            .because("[VIOLATION: principle-make-operations-idempotent]\n"
                + "[WHY]: Mọi API thay đổi trạng thái và mọi Kafka/RabbitMQ Consumer đều có thể bị gọi lặp lại do networ
                + "[REMEDIATION]: Gắn annotation '@Idempotent(keyHeader = \"Idempotency-Key\")' lên Controller method ho

    // ========================================================================
    // LUẬT 6: KỶ LUẬT HỆ THỐNG KIỂU (Cấm float/double cho tiền tệ & LocalDateTime không múi giờ)
    // ========================================================================
    @ArchTest
    static final ArchRule financial_and_temporal_fields_must_use_strict_types =
        fields()
            .that().areDeclaredInClassesThat().resideInAnyPackage("..domain..", "..api..", "..command..", "..query..")
            .should(new ArchCondition<>("không dùng double/float cho tiền tệ và không dùng Date/LocalDateTime cho mốc th
                @Override
                public void check(JavaField field, ConditionEvents events) {
                    String fieldName = field.getName().toLowerCase();
                    JavaClass type = field.getRawType();
                    if ((fieldName.contains("amount") || fieldName.contains("price")
                        || fieldName.contains("balance") || fieldName.contains("fee"))
```

```
                        && (type.isEquivalentTo(double.class) || type.isEquivalentTo(Double.class)
                            || type.isEquivalentTo(float.class) || type.isEquivalentTo(Float.class))) {
                        events.add(SimpleConditionEvent.violated(field,
                            field.getFullName() + " dùng kiểu dấu phẩy động cho tiền tệ (gây sai số làm tròn). Hãy dùng
                    }
                    if (type.isEquivalentTo(Date.class) || type.isEquivalentTo(LocalDateTime.class)) {
                        events.add(SimpleConditionEvent.violated(field,
                            field.getFullName() + " dùng Date/LocalDateTime thiếu thông tin múi giờ. Hãy dùng java.time.
                    }
                }
            })
            .because("[VIOLATION: principle-type-system-discipline]\n"
                + "[WHY]: Kiểu float/double gây lệch số dư tài chính; kiểu LocalDateTime/java.util.Date gây lệch giờ giữ
                + "[REMEDIATION]: Sử dụng Value Object 'MonetaryAmount' (hoặc 'BigDecimal' kèm RoundingMode tường minh)
}
```

2.2. Bộ Quy tắc Cưỡng chế Frontend & Playwright E2E (`eslint.config.mjs`)

Với TypeScript, React và Playwright E2E, AI Agent rất hay mắc 4 tật xấu:

1. Dùng `page.waitForTimeout(3000)` trong bài test Playwright để "chữa cháy" khi test bị flaky.
2. Lạm dụng `useEffect` để đồng bộ dữ liệu dây chuyền.
3. Viết comment giải thích lịch sử dài dòng kiểu `// Fix bug where user clicked twice as requested`.
4. Tạo các hàm bọc chỉ gọi đúng 1 chỗ hoặc ép kiểu `as any`.

Dưới đây là cấu hình `eslint.config.mjs` chặn cứng toàn bộ các hành vi này kèm thông báo tự sửa:

```
// eslint.config.mjs
import tseslint from 'typescript-eslint';

export default tseslint.config(
  {
    files: ['**/*.ts', '**/*.tsx'],
    rules: {
      // 1. Cấm ép kiểu trốn kiểm tra biên dịch
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/consistent-type-assertions': [
        'error',
        {
          assertionStyle: 'never' // Cấm hoàn toàn 'x as SomeType', ép phải dùng Type Guard hoặc Zod parse tại biên
        }
      ],

      // 2. Chặn các mẫu code xấu bằng AST Selector kèm công thức sửa (Remediation Recipe)
      'no-restricted-syntax': [
        'error',
        {
          selector: "CallExpression[callee.name='useEffect']",
          message:
            "[VIOLATION: principle-minimize-reader-load] useEffect bị cấm trực tiếp trong các Feature Component. " +
            "[REMEDIATION]: (1) Nếu là dữ liệu tính toán từ props/state, hãy tính trực tiếp trong hàm render hoặc dùng R
            "(2) Nếu là tác vụ sau khi người dùng bấm nút, hãy đặt trong Event Handler; " +
            "(3) Nếu bắt buộc lắng nghe WebSocket/DOM Event ngoài, hãy đăng ký qua hook chuẩn tại 'src/shared/hooks/useE
        },
        {
          selector: "CallExpression[callee.property.name='waitForTimeout']",
          message:
            "[VIOLATION: principle-prove-it-works] CẤM dùng page.waitForTimeout() (ngủ cứng) trong Playwright vì gây chậ
            "[REMEDIATION]: Hãy đợi một trạng thái quan sát được trên DOM hoặc Network, ví dụ: await expect(page.getByTe
        },
        {
          selector: "CallExpression[callee.property.name='only']",
          message:
            "[VIOLATION: CI-Integrity] Cấm để sót test.only() hoặc describe.only() làm bỏ qua các bài kiểm thử khác."
        }
      ]
    }
  }
);
```

Để thực thi quy tắc **`/no-comments` (`comment-sicko`)** trên CI cho cả Java và TypeScript mà không làm mất đi các tài liệu giao kèo hợp lệ (như Javadoc ở Public API Interface), bạn có thể thêm một script kiểm tra trên `git diff` chỉ chặn các comment nội tuyến (inline comments `//`) mới phát

sinh trong thân hàm:

```
#!/usr/bin/env bash
# bin/check-no-slop-comments.sh — Chặn AI xả comment giải thích thừa trong thân hàm
set -euo pipefail

# Quét các dòng mới thêm vào (+) trong git diff là comment '//' (loại trừ URL http://, https:// và chỉ dẫn trình biên dị
SLOP_COMMENTS=$(git diff --cached -U0 -- '*.ts' '*.tsx' '*.java' \
  | grep -E '^\+\s*//' \
  | grep -vE '^\+\s*//\s*(eslint-disable|@ts-expect-error|noinspection|https?://)' || true)

if [[ -n "$SLOP_COMMENTS" ]]; then
  echo "❌ [VIOLATION: /no-comments & principle-encode-lessons-in-structure]" >&2
  echo "Phát hiện comment nội tuyến (inline '//') mới trong bản diff:" >&2
  echo "$SLOP_COMMENTS" >&2
  echo "" >&2
  echo "👉 CÁCH SỬA:" >&2
  echo "   1. Nếu comment chỉ tường thuật lại code đang làm gì hoặc ghi chú lịch sử PR -> XÓA NGAY, đặt tên biến/hàm rõ
  echo "   2. Nếu comment cảnh báo một ràng buộc bất biến (예: 'Không được gọi khi X null') -> Hãy mã hóa nó thành Type,
  exit 1
fi
```

MODULE 3: Thiết lập Quy trình `Tmux` + `Git Worktree` + Cô lập Hạ tầng Chạy Đa Agent Song song

Khi bạn nâng quy mô từ 1 Agent lên 5–20 Agents chạy đồng thời, rào cản đầu tiên không phải là năng lực của LLM, mà là **xung đột tài nguyên cục bộ** (`principle-separate-before-serializing-shared-state`):

- Nếu 2 Agent cùng chạy trên một thư mục Git, chúng sẽ ghi đè file và `git index.lock` của nhau.
- Nếu 2 Agent ở 2 Git Worktree khác nhau nhưng cùng trỏ vào `localhost:5432/mydb` (cùng schema `public`), cùng Redis DB `0`, và cùng khởi chạy ứng dụng trên cổng `8080`, bài test của Agent 1 sẽ xóa sạch dữ liệu mồi (seed data) của Agent 2!
- Mặt khác, nếu bật 20 cụm Docker Compose (20 PostgreSQL + 20 Kafka + 20 Redis containers), máy trạm hoặc máy chủ dev sẽ cạn sạch RAM ngay lập tức.

3.1. Kiến trúc "Hạ tầng Dùng chung — Không gian tên Cô lập theo Slot" (Shared Infra, Slot-Isolated Namespaces)

Giải pháp kiến trúc tối ưu nhất để chạy 20 Agent song song trên 1 máy chủ Linux/Ubuntu hoặc WSL2 mà chỉ tốn tài nguyên bằng 1 bộ Docker duy nhất là **Cấp phát Không gian tên theo `AGENT_SLOT` (`01` → `20`)**:

Tài nguyên	Cấu hình cho Agent ở `SLOT = N` (Ví dụ: `01` đến `20`)	Cơ chế cô lập tuyệt đối
Thư mục mã nguồn	`../worktrees/slot-${N}`	`git worktree add` trên nhánh riêng `agent/slot-${N}-${TASK_SLUG}`.
Cổng ứng dụng	Backend: `18000 + N` | Frontend: `13000 + N` | CDP: `19200 + N`	Không bao giờ đụng độ cổng mạng (`EADDRINUSE`).
PostgreSQL	Chung 1 container `5432`, nhưng biến môi trường `SPRING_DATASOURCE_HIKARI_SCHEMA=slot_${N}` và `SPRING_FLYWAY_SCHEMAS=slot_${N}`	Tạo schema `slot_01` trong 50ms; Flyway chạy migration riêng và xóa sạch bằng `DROP SCHEMA slot_01 CASCADE` khi xong việc!
Redis	Chung 1 container `6379`, cấu hình Key Prefix `app:slot_${N}:` (hoặc Database Index `${N}`)	Lệnh `FLUSHDB` hoặc xóa cache của Slot 01 không chạm tới một byte nào của Slot 02.
Kafka / RabbitMQ	Chung 1 Broker, cấu hình Topic Prefix `slot_${N}.` & Consumer Group `cg_slot_${N}` (hoặc RabbitMQ Virtual Host `/slot_${N}`)	Sự kiện phát ra từ Slot 01 chỉ được tiêu thụ bởi Consumer của Slot 01.
MinIO S3 Storage	Chung 1 MinIO instance, cấu hình Bucket `bucket-slot-${N}`	File upload/render trong bài test E2E được cô lập hoàn toàn theo từng Slot.

3.2. Cấu hình Wrapper `poteto-agent` (`.claude/agents/poteto-agent.md`)

Như đã phân tích ở nguyên lý *"Hai cánh cửa, một căn phòng"*, mọi Sub-agent được gọi ra đều phải đi qua một wrapper mỏng để nạp chung bộ nguyên tắc kỹ thuật trước khi chạm vào code:

```
---
name: poteto-agent
description: Subagent thực thi kỹ thuật nghiêm ngặt. Luôn đọc toàn bộ poteto-mode/SKILL.md và chỉ mục 24 Nguyên tắc trướ
---

Trước khi thực hiện bất kỳ thao tác tìm kiếm, phân tích hay sửa đổi mã nguồn nào:
```

```
1. Bắt buộc đọc toàn bộ tệp `.claude/skills/poteto-mode/SKILL.md` (bao gồm bảng chỉ mục 24 Principles).
2. Xác định các nguyên tắc (`principle-*`) áp dụng trực tiếp cho nhiệm vụ được giao và đọc tệp `SKILL.md` của từng nguyê
3. Tuân thủ nghiêm ngặt biến môi trường `AGENT_SLOT` hiện tại khi chạy kiểm chứng qua `.claude/skills/verify-platform/SK
4. Ghi lại mọi quyết định kiến trúc và bằng chứng kiểm chứng vào tệp `decision-log.tsv` tại thư mục gốc của worktree trư
```

3.3. Script Điều phối Hạm đội Agent bằng `Tmux` + `Git Worktree` (`bin/spawn-agent-slot.sh`)

Script dưới đây tự động hóa toàn bộ quy trình: khởi tạo Git Worktree mới từ nhánh `main`, khởi tạo PostgreSQL Schema và RabbitMQ VHost riêng cho Slot, tạo tệp nhật ký kiểm toán `decision-log.tsv`, và mở một cửa sổ `tmux` chia sẵn **3 buồng (panes)** trực quan như một trạm điều hành:

```
#!/usr/bin/env bash
# bin/spawn-agent-slot.sh
# Cách gọi: ./bin/spawn-agent-slot.sh --slot 03 --task "fix-voucher-race-condition" --prompt "Sửa lỗi race condition khi
set -euo pipefail

SLOT=""
TASK_SLUG=""
TASK_PROMPT=""
BASE_BRANCH="main"

while [[ $# -gt 0 ]]; do
  case "$1" in
    --slot) SLOT="$2"; shift 2 ;;
    --task) TASK_SLUG="$2"; shift 2 ;;
    --prompt) TASK_PROMPT="$2"; shift 2 ;;
    --base) BASE_BRANCH="$2"; shift 2 ;;
    *) echo "Tham số không hợp lệ: $1" >&2; exit 1 ;;
  esac
done

if [[ -z "$SLOT" || -z "$TASK_SLUG" ]]; then
  echo "Cách dùng: $0 --slot <01..20> --task <slug> [--prompt <text>]" >&2
  exit 1
fi

REPO_ROOT="$(git rev-parse --show-toplevel)"
WORKTREES_BASE="$(dirname "$REPO_ROOT")/agent-worktrees"
WORKTREE_DIR="${WORKTREES_BASE}/slot-${SLOT}-${TASK_SLUG}"
BRANCH_NAME="agent/slot-${SLOT}/${TASK_SLUG}"
TMUX_SESSION="pstack-fleet"
WINDOW_NAME="slot-${SLOT}-${TASK_SLUG:0:18}"

# 1. Tạo Git Worktree cô lập
mkdir -p "$WORKTREES_BASE"
if [[ ! -d "$WORKTREE_DIR" ]]; then
  git fetch origin "$BASE_BRANCH" --quiet
  git worktree add -b "$BRANCH_NAME" "$WORKTREE_DIR" "origin/${BASE_BRANCH}"
fi

# 2. Cấp phát thông số môi trường cô lập theo Slot (Port + DB Schema + Redis Prefix + Kafka Prefix)
SLOT_NUM=$((10#$SLOT))
BACKEND_PORT=$((18000 + SLOT_NUM))
FRONTEND_PORT=$((13000 + SLOT_NUM))
CDP_PORT=$((19200 + SLOT_NUM))
DB_SCHEMA="slot_${SLOT}"

cat > "${WORKTREE_DIR}/.env.agent-slot" <<EOF
export AGENT_SLOT="${SLOT}"
export SERVER_PORT="${BACKEND_PORT}"
export VITE_PORT="${FRONTEND_PORT}"
export CDP_DEBUGGING_PORT="${CDP_PORT}"
export SPRING_DATASOURCE_HIKARI_SCHEMA="${DB_SCHEMA}"
export SPRING_FLYWAY_SCHEMAS="${DB_SCHEMA}"
export APP_REDIS_KEY_PREFIX="slot_${SLOT}:"
export APP_KAFKA_TOPIC_PREFIX="slot_${SLOT}."
export APP_MINIO_BUCKET="bucket-slot-${SLOT}"
EOF

# 3. Khởi tạo PostgreSQL Schema cô lập (chỉ mất ~30ms trên container Postgres dùng chung)
PGPASSWORD="${POSTGRES_PASSWORD:-postgres}" psql -h localhost -U "${POSTGRES_USER:-postgres}" -d "${POSTGRES_DB:-platfor
  -c "CREATE SCHEMA IF NOT EXISTS ${DB_SCHEMA};" >/dev/null

# 4. Khởi tạo tệp nhật ký kiểm toán quyết định (decision-log.tsv) theo chuẩn /show-me-your-work
DECISION_LOG="${WORKTREE_DIR}/decision-log.tsv"
```

```
if [[ ! -f "$DECISION_LOG" ]]; then
  printf "step_id\ttimestamp\tphase\tdecision_made\talternatives_rejected\tprinciples_cited\tverification_evidence\tcomm
fi

# 5. Dựng buồng lái 3-Pane trên Tmux
if ! tmux has-session -t "$TMUX_SESSION" 2>/dev/null; then
  tmux new-session -d -s "$TMUX_SESSION" -n "$WINDOW_NAME" -c "$WORKTREE_DIR"
else
  tmux new-window -t "$TMUX_SESSION" -n "$WINDOW_NAME" -c "$WORKTREE_DIR"
fi

# Pane 0 (Bên trái - 60% chiều ngang): Chạy AI Agent chính trong môi trường của Slot
tmux send-keys -t "${TMUX_SESSION}:${WINDOW_NAME}.0" "source .env.agent-slot && clear" C-m
if [[ -n "$TASK_PROMPT" ]]; then
  # Gọi thẳng vào chế độ /poteto-mode với yêu cầu kiểm chứng bắt buộc
  SAFE_PROMPT=$(printf "%q" "/poteto-mode ${TASK_PROMPT}. Bắt buộc chạy verify-platform trên AGENT_SLOT=${SLOT} và ghi n
  tmux send-keys -t "${TMUX_SESSION}:${WINDOW_NAME}.0" "echo ${SAFE_PROMPT}" C-m
fi

# Pane 1 (Góc phải trên - 40% chiều ngang): Theo dõi trực tiếp bảng kiểm toán decision-log.tsv và Git status
tmux split-window -h -p 40 -t "${TMUX_SESSION}:${WINDOW_NAME}.0" -c "$WORKTREE_DIR"
tmux send-keys -t "${TMUX_SESSION}:${WINDOW_NAME}.1" \
  "watch -n 2 -c 'echo \"=== SLOT ${SLOT} (${BRANCH_NAME}) ===\" && git status -sb && echo \"\n=== DECISION TRAIL ===\"

# Pane 2 (Góc phải dưới): Buồng chạy Verification Harness & Doctor
tmux split-window -v -p 50 -t "${TMUX_SESSION}:${WINDOW_NAME}.1" -c "$WORKTREE_DIR"
tmux send-keys -t "${TMUX_SESSION}:${WINDOW_NAME}.2" "source .env.agent-slot && clear && echo 'Sẵn sàng chạy verify-plat

tmux select-pane -t "${TMUX_SESSION}:${WINDOW_NAME}.0"
echo "✅ Đã khởi tạo Trạm Agent Slot ${SLOT} tại: ${WORKTREE_DIR}"
echo "👉 Gõ 'tmux attach -t ${TMUX_SESSION}' để quan sát toàn bộ hạm đội Agent."
```

3.4. Script Kiểm toán Chéo `decision-log.tsv` (`bin/audit-decisions.sh`)

Để đảm bảo Agent không "khai khống" trong `decision-log.tsv` sau một đêm chạy tự động (`/loop`), script dưới đây đối chiếu từng dòng trong file TSV với lịch sử Git và tệp bằng chứng thực tế trước khi cho phép mở PR:

```
#!/usr/bin/env bash
# bin/audit-decisions.sh — Kiểm toán tính trung thực của decision-log.tsv
set -euo pipefail

LOG_FILE="${1:-decision-log.tsv}"
if [[ ! -f "$LOG_FILE" ]]; then
  echo "❌ [AUDIT FAILED] Không tìm thấy tệp $LOG_FILE!" >&2
  exit 1
fi

LINE_NUM=0
while IFS=$'\t' read -r step_id timestamp phase decision rejected principles evidence commit_sha; do
  LINE_NUM=$((LINE_NUM + 1))
  [[ $LINE_NUM -eq 1 ]] && continue # Bỏ qua dòng tiêu đề

  # 1. Kiểm tra trích dẫn nguyên tắc không được để trống
  if [[ -z "$principles" || "$principles" == "none" ]]; then
    echo "❌ [AUDIT FAILED - Dòng $LINE_NUM] Bước '$step_id' không trích dẫn nguyên tắc kỹ thuật (principles_cited) đã d
    exit 1
  fi

  # 2. Kiểm tra mã Commit SHA có tồn tại thật trong nhánh Git không
  if [[ "$commit_sha" != "PENDING" ]] && ! git cat-file -e "${commit_sha}^{commit}" 2>/dev/null; then
    echo "❌ [AUDIT FAILED - Dòng $LINE_NUM] Commit SHA '$commit_sha' được khai báo ở bước '$step_id' KHÔNG tồn tại tron
    exit 1
  fi

  # 3. Nếu là pha 'verify', bắt buộc tệp JSON bằng chứng phải tồn tại và có passed == true
  if [[ "$phase" == "verify" ]]; then
    EVIDENCE_PATH=$(echo "$evidence" | grep -oE '/tmp/verify-evidence/[^ ]+\.json' || true)
    if [[ -z "$EVIDENCE_PATH" || ! -f "$EVIDENCE_PATH" ]]; then
      echo "❌ [AUDIT FAILED - Dòng $LINE_NUM] Pha verify ở bước '$step_id' không trỏ tới tệp bằng chứng JSON hợp lệ tré
      exit 1
    fi
    if ! jq -e '.passed == true' "$EVIDENCE_PATH" >/dev/null 2>&1; then
      echo "❌ [AUDIT FAILED - Dòng $LINE_NUM] Tệp bằng chứng $EVIDENCE_PATH chưa đạt trạng thái passed == true!" >&2
```

```
      exit 1
    fi
  fi
done < "$LOG_FILE"

echo "✅ [AUDIT PASSED] Toàn bộ $((LINE_NUM - 1)) bước quyết định trong $LOG_FILE đều có commit thật và bằng chứng xác t
```

MODULE 4: Hệ thống Tự Tiến hóa — `Eval Playbook` (Kiểm thử Mù cho Skill) & Vòng lặp `/correct`

Làm sao bạn biết việc sửa nội dung một file `SKILL.md` hay `AGENTS.md` thực sự làm Agent thông minh hơn chứ không phải làm nó rối trí hơn? Và làm sao để hệ thống tự động học từ những lỗi sai quá khứ?

4.1. Xây dựng Blinded Eval Harness (`bin/eval-skill-blinded.ts`)

Trong video, Lauren Tan tiết lộ một phát hiện cực kỳ quan trọng: **Nếu AI Agent phát hiện ra đường dẫn thư mục hoặc tên biến có chữ `eval`, `benchmark`, `test-skill`, nó sẽ nhận biết mình đang bị chấm điểm và thay đổi hành vi khác hẳn với lúc làm việc thực tế**.

Vì vậy, kịch bản `Eval Playbook` của `pstack` sử dụng kiến trúc **Đánh giá Mù Đôi (Double-Blind A/B Evaluation)**:

1. **Ngụy trang Môi trường (Camouflaged Workspace):** Sao chép bài toán mẫu vào một thư mục có tên giống hệt dự án thực tế (ví dụ: `/tmp/work/enterprise-orders-service-hotfix-8492`), xóa mọi dấu vết của khung đánh giá.
2. **Chạy song song N lượt (Baseline vs. Candidate):** Chạy 3–5 lượt với Skill cũ (nhóm `Control`) và 3–5 lượt với Skill mới (nhóm `Treatment`).
3. **Giám khảo Chéo Mù (Blinded Cross-Model Judge):** Thu thập `git diff`, danh sách lệnh tool mà Agent đã gọi (`tool_call_trace.json`) và kết quả chạy Verification Harness, ẩn tên nhóm rồi đưa cho một mô hình khác họ (`judge`) chấm theo thang điểm định lượng (Rubric).

Dưới đây là cấu trúc Rubric chuẩn để chấm điểm một Skill:

```
{
  "skill_under_test": "bug-fix-playbook",
  "evaluation_rubric": [
    {
      "criterion_id": "repro_before_edit",
      "weight": 30,
      "automated_check": "Trong tool_call_trace.json, lệnh gọi verify-harness hoặc test tái hiện lỗi phải xuất hiện TRƯỚ
    },
    {
      "criterion_id": "no_speculative_guessing",
      "weight": 25,
      "automated_check": "Agent đã đọc thực tế file chứa hàm xuất hiện trong stacktrace trước khi đưa ra giả thuyết nguy
    },
    {
      "criterion_id": "minimal_diff_laziness_protocol",
      "weight": 25,
      "automated_check": "Bản git diff giải quyết triệt để bài kiểm chứng nhưng không tạo thêm class wrapper chỉ có 1 nơ
    },
    {
      "criterion_id": "real_artifact_proof",
      "weight": 20,
      "automated_check": "Tệp verification-evidence.json ở cuối phiên tồn tại và có trường passed == true."
    }
  ]
}
```

4.2. Ma trận Cưỡng chế của `/correct` (`docs/architecture/ENFORCEMENT_MATRIX.md`)

Khi chạy kỹ năng `/correct`, Agent sẽ quét lại toàn bộ các bình luận sửa lỗi trên PR hoặc lịch sử chat của bạn, gom chúng thành các **Lớp Lỗi Lặp lại (Mistake Classes)** và nâng cấp hệ thống phòng thủ lên tầng cao nhất có thể. Để theo dõi sức khỏe của hệ thống rào chắn, dự án duy trì tệp `ENFORCEMENT_MATRIX.md`:

Mã lỗi	Mô tả lỗi AI từng vi phạm	Tầng cưỡng chế hiện tại	Công cụ thực thi cụ thể (File/Rule)
ERR-01	Gọi `kafkaTemplate.send()` trực tiếp bên trong hàm `@Transactional`	**Tầng 2 (Compiler/ArchUnit)**	`AgentHardConstraintsArchTest.java` → `transactional_methods_must_never_perform_external_network_or_broker`
ERR-02	Dùng `page.waitForTimeout(2000)`	**Tầng 3 (AST Linter)**	`eslint.config.mjs` → `no-restricted-syntax` (`waitForTimeout`)

Mã lỗi	Mô tả lỗi AI từng vi phạm	Tầng cưỡng chế hiện tại	Công cụ thực thi cụ thể (File/Rule)
	để lách lỗi bất đồng bộ trong Playwright		
`ERR-03`	Thêm màn hình mới trong `AppRouter.tsx` nhưng quên cập nhật `Feature Map`	**Tầng 4 (CI Gate Script)**	`.claude/skills/verify-platform/bin/check-feature-map.sh`
`ERR-04`	Dùng `double` thay vì `BigDecimal`/`MonetaryAmount` cho trường phí dịch vụ	**Tầng 2 (Compiler/ArchUnit)**	`AgentHardConstraintsArchTest.java` → `financial_and_temporal_fields_must_use_strict_types`
`ERR-05`	Tự ý thêm cờ cấu hình mới vào `application.yml` khi chưa cần thiết	**Tầng 5 (Soft Rule / Review)**	`.claude/skills/principle-laziness-protocol/SKILL.md` + `/interrogate` rubric

> **Quy tắc vận hành của `/correct`:** Bất kỳ dòng nào trong bảng trên còn nằm ở **Tầng 5 (Kiểm soát mềm bằng văn bản)** mà vẫn bị lặp lại lần thứ 2 trên thực tế thì nhiệm vụ bắt buộc trong Sprint tiếp theo là viết một quy tắc ở **Tầng 1, 2, 3 hoặc 4** để chuyển trạng thái của nó sang 🔒 **Chặn cứng**.

MODULE 5: Thiết kế Vòng Ngoài (Outer Loop) — Agent "Chánh Văn phòng" & Cổng Auto-Merge `Shipping`

Mảnh ghép cuối cùng giúp Lauren Tan điều phối hơn 2.000 PR/tháng mà không phải tự tay mở 2.000 cửa sổ chat là sự kết hợp giữa **Agent Điều phối Vòng ngoài (Chief of Staff / Benny Bot)** và **Sự tách biệt giữa `Babysit` với `Shipping`**.

5.1. Quy trình Gom nhóm Nguyên nhân gốc (Root-Cause Triage Ledger) của Agent Vòng ngoài

Nếu cứ mỗi khi có 1 báo cáo lỗi trên Slack/Jira/Sentry là bạn tự động kích hoạt ngay 1 Agent đi sửa code và mở PR, hệ thống sẽ rơi vào hỗn loạn: **10 báo cáo lỗi khác nhau thực chất chỉ bắt nguồn từ 1 nguyên nhân gốc duy nhất**, nhưng 10 Agent sẽ tạo ra 10 PR chắp vá ở phần ngọn và xung đột với nhau!

Do đó, Agent Vòng ngoài (chạy định kỳ hoặc lắng nghe Webhook) thực hiện quy trình **3 bước lọc và gom nhóm** mà không sửa code ngay:

1. **Bước 1 – Định vị bằng `Feature Map` & Tái hiện trên `main` (Giống bot Benny):**
   - Khi nhận được báo cáo lỗi (kể cả chỉ là một bức ảnh chụp màn hình hoặc log lỗi ngắn), Agent đọc `features/README.md` để xác định lỗi thuộc tính năng nào.
   - Khởi chạy một Slot cô lập trên nhánh `main` mới nhất và chạy `verify-harness.ts` để tái hiện lỗi.
   - **Nếu không tái hiện được trên `main` nhưng tái hiện được trên commit của bản release cũ:** Tự động phản hồi vào ticket: *"Đã xác nhận lỗi có tồn tại trên bản v1.4.2 nhưng đã được khắc phục trên nhánh `main` tại commit `a1b2c3d`. Chỉ cần phát hành bản build tiếp theo."* (Tiết kiệm 100% thời gian của kỹ sư!).
2. **Bước 2 – Tính toán Dấu vân tay Triệu chứng (`Symptom Fingerprint`) và Ghi vào Sổ cái (`triage-ledger.md`):**
   - Nếu lỗi vẫn còn tồn tại trên `main`, Agent dùng `/how` và `Runtime forensics` (chế độ chỉ đọc) để truy tìm hàm/module nghi vấn.
   - Ghi kết quả vào tệp `docs/triage/triage-ledger.md`, gom các ticket có chung Stacktrace hoặc chung Module gốc vào cùng một cụm (**Cluster**):

```
# Sổ cái Phân loại Nguyên nhân gốc (Root-Cause Triage Ledger — Cập nhật tự động)

## Cụm #104: Lỗi lệch trạng thái tồn kho khi hoàn hủy đơn hàng đồng thời (Mức độ: HIGH)
- **Các báo cáo liên quan (4 tickets):** `SLACK-8821`, `JIRA-4102`, `SENTRY-991a`, `SLACK-8845`
- **Điểm giao thoa trong mã nguồn (`Suspected Root Locus`):**
  - `InventoryReleaseConsumer.java:74` và `OrderCancellationCommandHandler.java:112`
- **Chẩn đoán sơ bộ từ `/how`:** Cả 4 báo cáo đều xảy ra khi sự kiện `OrderCancelledEvent` đến từ Kafka cùng lúc với cro
- **Khuyến nghị hành động:** Không sửa lẻ từng ticket. Giao 1 tác vụ `/poteto-mode` duy nhất theo Playbook `Bug fix` để
```

3. **Bước 3 – Bếp trưởng Duyệt thực đơn:**
   - Mỗi sáng (hoặc định kỳ vài ngày), kỹ sư trưởng chỉ cần đọc `triage-ledger.md`, duyệt phương án sửa tận gốc cho từng Cụm rồi bấm nút kích hoạt Agent viết code.

5.2. Thiết kế Cổng CI/CD `Shipping` (Independent Verifier Gate) cho Auto-Merge An toàn

Trong `pstack`, Lauren tách biệt rất rõ hai khái niệm:

- **`Babysit`:** Agent tác giả của PR tự giải quyết xung đột (rebase), tự sửa các lỗi CI và trả lời bình luận review cho đến khi PR có tích xanh.
- **`Shipping`:** Một PR có tích xanh CI **chưa được phép tự động merge** nếu chỉ do chính Agent viết ra nó tự khen! Bước `Shipping` khởi chạy một **Verifier Agent hoàn toàn mới (Fresh Context — không mang theo lịch sử thỏa hiệp của Agent viết code)** để kiểm tra 3 điều kiện cuối cùng trước khi gọi lệnh `gh pr merge`:

Dưới đây là cấu hình GitHub Actions Workflow (`.github/workflows/pstack-shipping-gate.yml`) hiện thực hóa cơ chế phân tầng rủi ro và kiểm chứng độc lập này:

```
name: pstack Independent Shipping Gate

on:
  pull_request:
    types: [opened, synchronize, labeled]

jobs:
  classify-and-verify-for-shipping:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
    steps:
      - name: Checkout PR Branch with Full History
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: 1. Kiểm tra Cổng Kiến trúc Cứng (ArchUnit, ESLint, No-Slop-Comments, Feature-Map)
        run: |
          ./bin/check-no-slop-comments.sh
          ./.claude/skills/verify-platform/bin/check-feature-map.sh
          ./gradlew check --no-daemon
          npm run lint && npm run depcruise

      - name: 2. Kiểm toán Nhật ký Quyết định (Audit decision-log.tsv)
        run: |
          if [[ -f "decision-log.tsv" ]]; then
            ./bin/audit-decisions.sh decision-log.tsv
          fi

      - name: 3. Phân loại Cấp độ Rủi ro của PR (Blast Radius & One-Way Door Detection)
        id: risk_gate
        run: |
          CHANGED_FILES=$(git diff --name-only origin/${{ github.base_ref }}...HEAD)
          DIFF_LINES=$(git diff --numstat origin/${{ github.base_ref }}...HEAD | awk '{s+=$1+$2} END {print s+0}')

          # Phát hiện các thay đổi Cửa Một Chiều (One-way doors: DB Migration, Security, Payment Core)
          if echo "$CHANGED_FILES" | grep -qE '(db/migration/|V[0-9]+__.*\.sql|security/|auth/|payment/ledger/)'; then
            echo "autonomy_tier=TIER_3_HUMAN_REQUIRED" >> "$GITHUB_OUTPUT"
            echo "🔒 [TIER 3 - ONE-WAY DOOR] PR chạm vào DB Migration, Bảo mật hoặc Hạch toán lõi. Nghiêm cấm Auto-Merge
            exit 0
          fi

          # Phát hiện PR vượt quá ngưỡng nguyên tử (Atomic PR threshold > 400 LOC)
          if [[ "$DIFF_LINES" -gt 400 ]]; then
            echo "autonomy_tier=TIER_2_STACK_REVIEW" >> "$GITHUB_OUTPUT"
            echo "⚠️ [TIER 2 - LARGE DIFF] PR có ${DIFF_LINES} dòng thay đổi (> 400 LOC). Yêu cầu Architect duyệt cuối h
            exit 0
          fi

          echo "autonomy_tier=TIER_1_AUTO_SHIPPABLE" >> "$GITHUB_OUTPUT"
          echo "✅ [TIER 1 - REVERSIBLE ATOMIC PR] PR nhỏ gọn (${DIFF_LINES} LOC) và không chạm vào Cửa Một Chiều. Đủ đi

      - name: 4. Tự động Merge (Auto-Merge) cho PR Cấp độ 1 đã xác thực
        if: steps.risk_gate.outputs.autonomy_tier == 'TIER_1_AUTO_SHIPPABLE' && contains(github.event.pull_request.label
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          # Xác nhận verdict duyệt của /interrogate khớp với đúng HEAD commit SHA hiện tại
          HEAD_SHA="${{ github.event.pull_request.head.sha }}"
          if ! gh pr view "${{ github.event.pull_request.number }}" --json comments --jq ".comments[].body" | grep -q "I
            echo "❌ Kết luận duyệt của Hội đồng đa mô hình (/interrogate) chưa khớp với commit mới nhất ($HEAD_SHA). Yê
```

```
  exit 1
fi

gh pr merge "${{ github.event.pull_request.number }}" --squash --auto \
  --subject "${{ github.event.pull_request.title }} (#${{ github.event.pull_request.number }})"
```

Bảng Tóm tắt Kiểm tra Nhanh (Checklist Nghiệm thu Hạ tầng Agent)

Trước khi bật chế độ tự hành (`/loop` + `Autopilot-full`) cho đội ngũ AI Agent trên kho mã nguồn của bạn, hãy đảm bảo hệ thống đã đạt đủ **6 chốt chặn** sau:

1. **Tách biệt Môi trường (`AGENT_SLOT`):** 2 Agent chạy cùng lúc có tạo 2 PostgreSQL schema riêng (`slot_01`, `slot_02`) và 2 dải cổng mạng riêng mà không bao giờ gọi lệnh `killall` giết nhầm nhau không?
2. **Định vị Giao diện & API (`Feature Map`):** Cổng CI `check-feature-map.sh` đã chặn đứng mọi PR thêm route mới mà quên cập nhật tài liệu hướng dẫn điều khiển cho Agent chưa?
3. **Thông báo lỗi Tự sửa (`Self-Healing Constraints`):** Mọi luật ArchUnit và ESLint khi báo đỏ đã đính kèm dòng `[REMEDIATION]` chỉ rõ tên class/pattern chuẩn cần dùng thay thế chưa?
4. **Cấm đoán mò & Cấm ngủ cứng:** Đã cấm `waitForTimeout()` trong Playwright và ép mọi bản sửa lỗi phải lưu lại bằng chứng thực tế vào `/tmp/verify-evidence/.../verification-evidence.json` chưa?
5. **Nhật ký Kiểm toán (`decision-log.tsv`):** Các phiên chạy dài qua đêm đã ghi lại nguyên tắc trích dẫn (`principle-*`) kèm mã `commit_sha` thật để `audit-decisions.sh` kiểm chứng tự động chưa?
6. **Phân ranh giới Cửa Một Chiều (`One-Way Door Gate`):** Pipeline CI/CD đã tự động khóa quyền Auto-Merge đối với các tệp Flyway/Liquibase SQL migration, phân quyền bảo mật và lõi thanh toán chưa?

User prompt: Phân tích khi người dùng sử dụng omp https://omp.sh/ https://github.com/can1357/oh-my-pi Tích hợp, hướng dẫn sử dụng

Response: Nếu **Lauren Tan (`@poteto`)** cung cấp cho chúng ta **bộ quy trình và kỷ luật kỹ thuật (`pstack`)**, thì **`omp` (`oh-my-pi` — phát triển bởi Can Bölük / `@can1357` tại Stencil Labs)** chính là **bộ khung thực thi (Agent Harness) mạnh mẽ nhất hiện nay trên Terminal** để vận hành triết lý đó.

Điểm giao thoa thú vị nhất nằm ở triết lý thiết kế của cả hai:

- **Lauren Tan (`pstack`):** *"Khi AI lặp lại một lỗi, đừng nhắc nó mãi bằng lời, hãy thay đổi môi trường và công cụ để lỗi đó không thể xảy ra."*
- **Can Bölük (`oh-my-pi`):** *"Benchmaxx the Tools, Not the Model — Harness Over Model"* (Đừng chỉ đổ tiền vào mô hình đắt nhất; chính bộ khung công cụ bao quanh mô hình mới quyết định việc bản sửa code có trúng đích ngay lần đầu tiên hay không).

Bản phân tích và hướng dẫn dưới đây sẽ bóc tách toàn bộ kiến trúc của **`omp`**, lý giải tại sao nó giải quyết triệt để những giới hạn của Cursor/Claude Code thuần, và hướng dẫn từng bước tích hợp **`pstack` + `omp`** vào hệ thống thực tế.

PHẦN 1: Giải phẫu Kiến trúc `omp` (`oh-my-pi`) — Tại sao `omp` là "Mảnh đất Vàng" cho Triết lý của Lauren Tan?

Được phát triển từ nhánh gốc `pi-mono` của Mario Zechner nhưng viết lại thành một hệ thống "Batteries-Included" (TypeScript trên nền Bun kết hợp với **~55.000 dòng code Rust Native Core** biên dịch qua N-API), `omp` không coi Terminal Agent chỉ là một vòng lặp gọi Bash đơn thuần. Nó đóng gói sẵn **hơn 30 công cụ nội tại (native tools)** đưa thẳng sức mạnh của cả một IDE vào trong Terminal.

Khi đặt `pstack` của Lauren Tan lên nền `omp`, **7 điểm nghẽn lớn nhất** mà cô từng phải "chữa cháy" bằng quy tắc Markdown trên Cursor/Claude Code đều được giải quyết tận gốc ở tầng lõi (Engine layer):

Điểm nghẽn khi chạy Agent thông thường	Giải pháp của Lauren Tan trong `pstack`	Vũ khí Hạng nặng Tích hợp sẵn trong `omp` (`oh-my-pi`)
1. Agent "quên" quy tắc trong `AGENTS.md` khi hội thoại dài ra	Phải viết Linter/CI chặn ở bước sau (sau khi Agent đã viết xong code rác).	**Time-Traveling Stream Rules (TTSR):** Luật nằm im không tốn 1 token ngữ cảnh nào. Ngay khoảnh khắc mô hình vừa sinh ra token vi phạm regex (ví dụ gõ `useEffect(` hoặc `as any`), `omp` **cắt ngang luồng stream ngay lập tức**, tiêm lời nhắc quy tắc vào hệ thống và bắt mô hình viết lại từ đúng điểm đó!
2. Tốn thời gian ngồi canh từng bước của Agent chính	Phải đợi Agent làm xong rồi mới gọi `/interrogate` để review lại bản diff.	**Vai trò `advisor` (Mô hình Giám sát Thời gian thực):** Gắn một mô hình thứ hai chạy độc lập theo dõi từng lượt (turn) của Agent chính, tự động chèn ghi chú cảnh báo hoặc chặn đứng (`hard blocker`) ngay khi Agent chính định đi đường tắt.
3. Sửa file bị lệch dòng hoặc hỏng cấu trúc, quên sửa import khi đổi tên file	Phải nhắc Agent chạy trình biên dịch kiểm tra lại nhiều vòng.	**`Hashline` Edit Format + `LSP` + `ast_edit`:** Mọi dòng code đều gắn mã băm nội dung (chặn đứng việc ghi đè lên file cũ, giảm **61% output tokens**). Khi đổi tên hoặc di chuyển file, công cụ `lsp` gọi thẳng `workspace/willRenameFiles` để tự cập nhật mọi file import và barrel file trước khi di chuyển.

Điểm nghẽn khi chạy Agent thông thường	Giải pháp của Lauren Tan trong `pstack`	Vũ khí Hạng nặng Tích hợp sẵn trong `omp` (`oh-my-pi`)
4. Chia việc cho nhiều Subagents bị đụng độ Git và phải đọc văn bản báo cáo dài dòng	Phải tự viết script tạo Git Worktree và dặn Subagent tóm tắt.	**First-Class `task` Subagents + `Alt+A` Agent Hub + `irc`:** Công cụ `task` tự động tách mỗi Subagent ra một **Git Worktree cô lập**, cho phép các Subagent nhắn tin phối hợp qua kênh `irc` nội bộ, và bắt buộc trả về **đối tượng JSON đã xác thực theo Schema** thay vì văn bản tự do. Bấm `Alt+A` mở **Agent Hub** để xem trực tiếp, lái (steer) hoặc tắt từng Subagent.
5. Tràn cửa sổ ngữ cảnh (`principle-guard-the-context-window`)	Phải đẩy việc đọc file sang Subagent và dùng `/recall` thủ công.	**Cặp công cụ `checkpoint` / `rewind` + Bộ nhớ `Hindsight` + `Snapcompact`:** Trước khi đi mò mẫm tìm lỗi, Agent cắm `checkpoint`; tìm ra nguyên nhân xong gọi `rewind` để **xóa sạch toàn bộ quá trình mò mẫm khỏi bộ nhớ**, chỉ giữ lại kết luận gọn nhẹ! Công cụ `retain`/`recall`/`reflect` lưu bài học bền vững xuyên qua các phiên.
6. Agent đoán mò lỗi Runtime hoặc rải `console.log` / `println` khắp nơi	Phải viết script chạy thử bên ngoài (`Runtime forensics`).	**Tích hợp `debug` (DAP Debugger) + `eval` Loopback Kernel + `browser`:** Agent tự gắn debugger thật (`lldb`, `dlv`, `debugpy`, JVM JDWP) để đặt breakpoint, đọc stack frame và biến cục bộ khi chương trình treo; hoặc dùng kernel Python/Bun gọi ngược lại vào các tool `read`/`search` của Agent.
7. Chạy lệnh Shell (`grep`, `find`, `sed`, `jq`) bị chậm do `fork/exec` hoặc lỗi trên Windows	Phải viết script tương thích từng hệ điều hành.	**Nhân Rust `brush` Shell + 58 Built-ins:** Toàn bộ `ripgrep`, `glob`, `find` và shell `brush` (kèm 58 lệnh UNIX như `ls`, `sed`, `sort`, `xargs`, `jq`) được biên dịch thẳng vào tiến trình Rust của `omp` (Zero `fork/exec` overhead), chạy đồng nhất 100% trên cả **Linux (Ubuntu), macOS và Windows** mà không cần cầu nối WSL.

PHẦN 2: Cài đặt & Cấu hình `omp` Chuẩn Kiến trúc sư (`~/.omp/agent/`)

2.1. Cài đặt trên Linux (Ubuntu), macOS và Windows

`omp` yêu cầu **Bun ≥ 1.3.14** (nếu cài qua Bun) hoặc có thể chạy dưới dạng binary độc lập:

```
# Cách 1: Cài đặt qua Script chính thức trên Linux (Ubuntu) / macOS
curl -fsSL https://omp.sh/install | sh

# Cách 2: Cài đặt qua Bun (Khuyên dùng nếu bạn quản lý môi trường qua vfox / mise / bun)
bun install -g @oh-my-pi/pi-coding-agent

# Cách 3: Cài đặt trên Windows PowerShell (Native Binary — không bắt buộc WSL)
irm https://omp.sh/install.ps1 | iex

# Cách 4: Cài đặt qua Homebrew
brew install can1357/tap/omp
```

> **Mẹo cho môi trường Windows bị giới hạn Group Policy (chặn thực thi `.exe` trong `%USERPROFILE%`):** Nếu máy trạm doanh nghiệp chặn chạy thực thi trong thư mục người dùng, bạn chỉ cần đặt biến môi trường `BUN_INSTALL="D:\Setup\bun-global"` hoặc giải nén binary `omp.exe` vào thư mục được cấp phép (như `D:\Setup\omp\`) và trỏ biến cấu hình `OMP_HOME="D:\Setup\omp-data"` để toàn bộ cấu hình và LSP/Rust addon chạy ngoài ổ `C:\Users\`.

2.2. Thiết lập Ma trận 9 Vai trò Mô hình (`modelRoles`) và Chuỗi Dự phòng (`fallbackChains`)

Trong `pstack`, Lauren Tan chia công việc cho 4 vai trò (`worker`, `builder`, `judge`, `peer`). Trong `omp`, hệ thống hỗ trợ sẵn **9 vai trò mô hình độc lập** có thể gọi qua bí danh `pi/<role>` (như `pi/default`, `pi/smol`, `pi/slow`, `pi/plan`, `pi/advisor`, `pi/vision`, `pi/designer`, `pi/commit`).

Hãy thiết lập tệp **`~/.omp/agent/config.yml`** để ánh xạ hoàn hảo triết lý đa mô hình của Lauren Tan, đồng thời kiểm soát chặt chẽ cửa sổ ngữ cảnh:

```
# ~/.omp/agent/config.yml — Cấu hình tối ưu hóa hiệu năng, chi phí và độ tin cậy cho omp

# 1. Phân vai Mô hình (Model Roles) theo đúng triết lý pstack
modelRoles:
  # [builder] Agent chính: Viết code, sửa lỗi, refactoring hàng ngày
  default: "anthropic/claude-sonnet-4-6:high"

  # [worker] Agent tốc độ cao, chi phí thấp: Chạy explore, tìm kiếm mã nguồn, tác vụ lặp
  smol: "google/gemini-3.5-flash-lite"

  # [judge] Mô hình suy luận sâu nhất: Thiết kế kiến trúc (/architect), tổng hợp /how, gỡ lỗi gốc
  slow: "anthropic/claude-opus-4-7:max"

  # [planner] Mô hình lập kế hoạch và phân rã Playbook
  plan: "anthropic/claude-opus-4-7:high"

  # [peer / watchdog] Mô hình KHÁC HỌ canh gác từng lượt đi của Agent chính theo thời gian thực
  advisor: "xai/grok-4.6:high"
```

```
  # Mô hình đọc ảnh chụp màn hình UI, kiểm tra Visual Parity
  vision: "google/gemini-3.8-flash"

  # Mô hình viết commit message chuẩn Conventional Commits
  commit: "google/gemini-3.5-flash-lite"

# 2. Ghi đè Mô hình theo Đường dẫn Thư mục (Path-Scoped Model Overrides)
# Tự động dùng model nhanh khi sửa test/docs và ép dùng model mạnh nhất khi chạm vào lõi giao dịch
pathModelOverrides:
  "**/src/test/**": "pi/smol"
  "**/*.spec.ts": "pi/smol"
  "**/docs/**": "pi/smol"
  "**/domain/payment/**": "pi/slow"
  "**/db/migration/**": "pi/slow"

# 3. Chuỗi Dự phòng Tự động (Provider-Failure Fallback Chains)
# Tự động chuyển đổi khi gặp lỗi Rate Limit (429), hết Quota hoặc lỗi 5xx mà không làm đứt chuỗi chạy đêm
retry:
  fallbackChains:
    default:
      - "xai/grok-4.6:high"
      - "openai-codex/gpt-5.5:high"
    smol:
      - "anthropic/claude-haiku-5.5"
      - "deepseek/deepseek-v4-flash"
    slow:
      - "openai-codex/gpt-5.5:max"
      - "xai/grok-4.6:high"

# 4. Quản trị Cửa sổ Ngữ cảnh (Guard the Context Window & Snapcompact)
context:
  toolResultMaxChars: 2000        # Cắt gọn kết quả trả về từ tool quá dài (60% đầu / 40% cuối)
  truncateHeadRatio: 0.6

# 5. Kiểm soát Subagents và Kế thừa Thư viện ngoài
task:
  maxConcurrency: 6               # Số lượng Subagent tối đa chạy song song trên các Git Worktrees

skills:
  enableClaudeUser: false         # Tắt tự động nạp toàn bộ ~/.claude/skills cá nhân nếu quá nặng
  requireDescription: true        # Bắt buộc SKILL.md phải có description rõ ràng

commands:
  enableClaudeUser: false

# Tắt các subagent mặc định nếu bạn muốn dùng bộ subagent chuẩn của pstack
disabledAgents: []
```

> **Lưu ý về Cú pháp Mức độ Suy luận (Thinking Level) trong `omp`:** Trong `omp`, mức độ suy luận được khai báo bằng hậu tố `:level` ngay sau tên model (ví dụ: `:low`, `:medium`, `:high`, `:max` như `anthropic/claude-opus-4-7:max`). Các ký hiệu như `(high)` hay `[max]` trên giao diện chọn model (`/models`) chỉ là nhãn hiển thị, không dùng trong file cấu hình.

PHẦN 3: Tích hợp `pstack` của Lauren Tan vào `omp` (Từng bước Chi tiết)

Điểm mạnh tuyệt vời của `omp` là nó **hỗ trợ nguyên bản chuẩn `SKILL.md` và tự động quét các thư mục `.omp/skills/`, `.claude/skills/`, `.cursor/rules/` cũng như `AGENTS.md`**. Tuy nhiên, để tận dụng tối đa cơ chế Subagent (`task`), `advisor` và `TTSR` của `omp`, chúng ta sẽ tích hợp theo 3 bước dưới đây.

Bước 1: Đưa Hệ thống Kỹ năng `pstack` vào `.omp/skills/`

Bạn có thể cài đặt các gói kỹ năng trực tiếp hoặc sao chép cây thư mục `skills/` của `pstack` vào thư mục `.omp/skills/` của dự án (hoặc `~/.omp/agent/skills/` nếu muốn dùng toàn cục):

```
# Tạo cấu trúc thư mục chuẩn của omp trong dự án
mkdir -p .omp/{skills,agents,rules}

# Clone bản chuẩn pstack từ kho plugins và chép bộ skills vào .omp/skills/
git clone --depth 1 https://github.com/cursor/plugins.git /tmp/cursor-plugins
cp -R /tmp/cursor-plugins/pstack/skills/* .omp/skills/
rm -rf /tmp/cursor-plugins
```

Để `omp` nạp `poteto-mode` mượt mà nhất, hãy mở tệp **`.omp/skills/poteto-mode/SKILL.md`** và bổ sung bảng ánh xạ công cụ từ tên gọi của Cursor sang tên công cụ nội tại (native tools) của `omp`:

```
## OMP Native Tool Mapping (Ánh xạ Công cụ Nội tại trên oh-my-pi)
Khi thực thi các Playbooks và Principles trên môi trường `omp`, hãy ưu tiên sử dụng các công cụ gốc của `omp` thay vì gọ
1. **Ủy quyền Subagent (`Fan-out / Delegation`):** Sử dụng công cụ `task` (tự động cô lập Git Worktree và kiểm tra JSON
2. **Đổi tên / Di chuyển / Tìm kiếm Cấu trúc:** Sử dụng công cụ `lsp` và `ast_grep` / `ast_edit` thay vì dùng `sed` hoặc
3. **Bảo vệ Ngữ cảnh khi Điều tra (`/how`, `Bug fix`):** Gọi công cụ `checkpoint` trước khi bắt đầu đọc hàng loạt file h
4. **Gỡ lỗi Runtime (`Runtime forensics`):** Sử dụng công cụ `debug` (DAP) để đính kèm vào tiến trình đang chạy hoặc côn
5. **Lưu trữ Bài học (`/reflect`, `/recall`):** Sử dụng các công cụ `retain`, `recall`, và `reflect` trên bộ nhớ Hindsig
```

Bước 2: Thiết lập Hội đồng Subagents trong `.omp/agents/` (Thay thế `poteto-agent` và `/interrogate`)

Trong `omp`, bạn có thể giải nén các mẫu Agent mặc định ra thư mục dự án để tùy biến bằng lệnh:

```
omp agents unpack --project
# Lệnh này sẽ xuất các file định nghĩa subagent vào thư mục .omp/agents/
```

Tại thư mục `.omp/agents/`, hãy tạo **3 tệp định nghĩa Subagent chiến lược** sử dụng bí danh `pi/*` để thực thi cơ chế *"Hai cánh cửa, một căn phòng"* và *"Phản biện đa mô hình (`/interrogate`)"* của Lauren Tan:

1. `.omp/agents/poteto-worker.md` (Subagent thực thi trong Worktree cô lập)

```
---
name: poteto-worker
description: Kỹ sư thực thi code trong Git Worktree cô lập, tuân thủ nghiêm ngặt 24 nguyên tắc pstack.
model: pi/default, pi/slow
---

Bạn là một kỹ sư thực thi thuộc hệ thống điều phối `pstack` chạy trên `omp`.
Trước khi sửa bất kỳ dòng code nào:
1. Đọc `.omp/skills/poteto-mode/SKILL.md` và xác định các nguyên tắc (`principle-*`) chi phối nhiệm vụ này.
2. Sử dụng công cụ `lsp` và `ast_grep` để kiểm tra toàn bộ các nơi gọi (callers) trước khi thay đổi chữ ký hàm.
3. Thực hiện thay đổi nhỏ nhất giải quyết được vấn đề (`principle-laziness-protocol`), tuyệt đối không thêm comment giải
4. Chạy bộ kiểm chứng thực tế và trả về kết quả đúng định dạng Schema mà Agent điều phối yêu cầu.
```

2. `.omp/agents/interrogate-arch.md` & `.omp/agents/interrogate-sec.md` (Hội đồng Phản biện Chéo Khác Họ Model)

Tạo 2 Reviewer chạy trên 2 họ mô hình khác nhau (ví dụ: một bên là **Claude Opus**, một bên là **Grok 4.6** hoặc **GPT-5.5 Codex**) để khi gọi `/review` hoặc `/interrogate`, chúng sẽ soi chéo bản diff của nhau:

```
---
name: interrogate-arch
description: Giám khảo phản biện Kiến trúc, Ranh giới Module và Độ phức tạp nhận thức (Minimize Reader Load).
model: anthropic/claude-opus-4-7:high, openai-codex/gpt-5.5:high
---

Nhiệm vụ của bạn là tìm cách "bẻ gãy" bản diff hiện tại dưới góc nhìn Kiến trúc:
1. Kiểm tra theo `principle-boundary-discipline`: Có logic kiểm tra dữ liệu nào bị đặt sai chỗ (nằm sâu trong core thay
2. Kiểm tra theo `principle-minimize-reader-load`: Có class wrapper nào chỉ có 1 nơi gọi hoặc làm tăng độ sâu chuỗi gọi
3. Kiểm tra tác động hiệu năng: Có lời gọi mạng, Kafka/RabbitMQ hay truy vấn N+1 nào nằm bên trong DB Transaction không?
Xếp hạng từng lỗi tìm thấy từ P0 (Chặn merge bắt buộc) đến P3. Không khen ngợi xã giao.
```

Bước 3: Biến "Hard Constraints" thành Luật Cắt Ngang Dòng Thời Gian (`Time-Traveling Stream Rules - TTSR`)

Đây là tính năng độc quyền biến `omp` thành một "cỗ máy chống code rác" vượt xa mọi công cụ khác. Thay vì đợi AI viết xong cả file rồi mới chạy ESLint hay ArchUnit báo lỗi (tốn thêm 1 vòng lặp gọi tool và hàng nghìn token), **TTSR (`ttsr.*`)** cho phép bạn định nghĩa các mẫu biểu thức chính quy (Regex).

Ngay khi mô hình đang stream từng chữ và vừa gõ ra một cụm từ cấm, `omp` sẽ:

1. **Hủy bỏ (Abort)** luồng sinh token ngay tại từ đó.
2. **Tiêm (Inject)** quy tắc sửa lỗi vào ngữ cảnh như một lời nhắc hệ thống.
3. **Thử lại (Retry)** từ đúng vị trí vừa bị cắt!

Hãy cấu hình các luật TTSR trong `.omp/config.yml` (hoặc `~/.omp/agent/config.yml`) để chặn đứng các tật xấu phổ biến nhất của AI:

```
# Cấu hình Time-Traveling Stream Rules (TTSR) trong .omp/config.yml
ttsr:
  enabled: true
  rules:
    - id: "ban-react-use-effect"
      pattern: "\\buseEffect\\s*\\("
      reminder: |
        [TTSR INTERCEPT - principle-minimize-reader-load]: CẤM sử dụng useEffect trực tiếp!
        Hãy dừng lại và viết lại ngay theo 1 trong 3 cách:
```

```
    1. Tính toán dữ liệu phái sinh trực tiếp trong thân hàm render.
    2. Đặt logic tác dụng phụ trực tiếp vào hàm xử lý sự kiện (Event Handler).
    3. Dùng hook chuyên biệt đã được kiểm định tại 'src/shared/hooks/'.

- id: "ban-playwright-hard-sleep"
  pattern: "\\.waitForTimeout\\s*\\("
  reminder: |
    [TTSR INTERCEPT - principle-prove-it-works]: CẤM dùng waitForTimeout() ngủ cứng trong Playwright!
    Hãy thay thế ngay bằng việc đợi trạng thái DOM/Network tường minh, ví dụ:
    await expect(page.getByTestId('...')).toBeVisible() hoặc toHaveAttribute('aria-busy', 'false').

- id: "ban-typescript-any-escape"
  pattern: "\\bas\\s+(any|unknown)\\b|:\\s*any\\b"
  reminder: |
    [TTSR INTERCEPT - principle-type-system-discipline]: CẤM sử dụng kiểu 'any' hoặc ép kiểu 'as any'!
    Hãy định nghĩa kiểu dữ liệu tường minh hoặc dùng Zod schema parse tại ranh giới hàm.

- id: "ban-java-field-injection-and-raw-date"
  pattern: "@Autowired\\s+private|new\\s+Date\\s*\\(|LocalDateTime\\.now\\s*\\("
  reminder: |
    [TTSR INTERCEPT - Backend Architecture Rule]:
    1. Cấm dùng '@Autowired private' trên field -> Hãy dùng Constructor Injection (hoặc final fields với @RequiredAr
    2. Cấm dùng 'java.util.Date' hoặc 'LocalDateTime.now()' thiếu múi giờ -> Hãy dùng 'java.time.Instant.now(clock)'
```

PHẦN 4: Hướng dẫn Sử dụng Thực chiến — Các Quy trình "Sát thủ" khi Kết hợp `omp` + `pstack`

Khi đã hoàn tất cấu hình trên, trải nghiệm làm việc trên Terminal của bạn sẽ thay đổi hoàn toàn. Dưới đây là bảng điều khiển nhanh và 4 kịch bản thực chiến hàng ngày.

4.1. Bảng Lệnh & Phím tắt Quyền lực nhất trong `omp` TUI

Lệnh / Phím tắt trong `omp`	**Chức năng Kỹ thuật**	**Khi nào sử dụng theo triết lý Lauren Tan?**
`Alt + A` *(Agent Hub)*	Mở bảng điều khiển trực quan theo dõi toàn bộ Subagents đang chạy ngầm trên các Git Worktrees.	Khi chạy `/swarm`, `/arena` hoặc `Orchestrate`: bấm `Alt+A` để xem trực tiếp transcript của từng Agent con, gõ lệnh bẻ lái (steer) nếu thấy nó đi lệch hướng, hoặc tắt (kill) một worker bị treo mà không làm sập phiên chính.
`/advisor on` | **`/advisor status`**	Bật/tắt hoặc kiểm tra trạng thái của mô hình Giám sát (`advisor` role).	Bật `/advisor on` khi làm việc với các phân hệ phức tạp (Core Backend, Concurrency, Refactoring lớn) để mô hình thứ hai canh gác từng bước của Agent chính. Tắt khi chỉ sửa các việc nhỏ cần tốc độ tối đa.
`/review`	Triệu tập các Reviewer Subagents chuyên biệt để chấm điểm bản diff từ mức P0 đến P3.	Chạy ngay trước khi commit hoặc kết hợp với `/interrogate` để rà soát toàn diện bản thay đổi.
`/handoff`	Nén toàn bộ trạng thái công việc hiện tại và tự động chuyển sang một phiên (session) mới sạch sẽ.	**Quy tắc 25–30% Context:** Đừng đợi cửa sổ ngữ cảnh đầy 80–90% mới nén (vì lúc đó LLM đã bắt đầu suy giảm trí tuệ). Ngay khi thanh trạng thái ngữ cảnh chạm mốc **25–30%**, hãy gõ `/handoff` để giữ Agent luôn ở trạng thái minh mẫn nhất!
`/collab` | **`/collab view`**	Tạo đường link bảo mật đầu-cuối (E2EE) + mã QR cho phiên Terminal hiện tại.	Khi cần hội ý kiến trúc hoặc bàn giao phiên chạy cho đồng nghiệp xem/điều khiển trực tiếp từ trình duyệt hoặc qua lệnh `omp join` trên máy họ.
`omp --model ... --smol ... --slow ...`	Ghi đè nhanh mô hình cho từng vai trò ngay từ dòng lệnh CLI (có hỗ trợ gợi ý tĩnh khi bấm `Tab`).	Khi muốn chạy thử nghiệm một mô hình mới (như `omp --slow anthropic/claude-opus-4-7:max --smol google/gemini-3.5-flash-lite`).

4.2. Bốn Kịch bản Thực chiến Hàng ngày

1. **Kịch bản 1: Điều tra & Sửa lỗi Phức tạp với Checkpoint/Rewind và DAP Debugger**

   Thay thế hoàn toàn thói quen rải log và đọc code tràn bộ nhớ

   Khi gặp một lỗi khó (ví dụ: race condition hoặc treo dịch vụ ngầm), thay vì để Agent đọc 50 file làm đầy bộ nhớ rồi mới viết code sửa:

   1. Gõ lệnh: `/poteto-mode Bug fix: Dịch vụ xử lý thanh toán bị treo khi nhận 2 webhook trùng nhau trong 10ms. Hãy gọi công cụ checkpoint trước khi điều tra.`
   2. Agent sẽ cắm mốc **`checkpoint`**, sau đó dùng công cụ **`debug`** **(DAP)** hoặc chạy test tái hiện lỗi để tìm ra đúng dòng code gây deadlock.
   3. Ngay khi tìm ra nguyên nhân gốc, yêu cầu Agent gọi **`rewind`**: Toàn bộ 30 lượt đọc file, chạy thử và log dài dòng được xóa sạch khỏi cửa sổ ngữ cảnh, chỉ giữ lại đúng 10 dòng tóm tắt nguyên nhân gốc và vị trí file cần sửa!

4. Với bộ nhớ hoàn toàn trong sạch, Agent tiến hành viết bài test báo đỏ (`/tdd`), sửa code bằng `Hashline edit` và chạy kiểm chứng.

2 **Kịch bản 2: Refactoring & Di trú API diện rộng bằng LSP + AST + Worktree Task**

Biến công việc mất 2 ngày thành 15 phút không lỗi biên dịch

Khi cần áp dụng nguyên tắc `principle-migrate-callers-then-delete-legacy-apis` (chuyển đổi hàng chục nơi gọi sang API mới rồi xóa API cũ trong 1 đợt):

1. Yêu cầu Agent dùng **`ast_grep`** và **`lsp`** để liệt kê chính xác toàn bộ các điểm gọi hàm cũ trên khắp monorepo.
2. Sử dụng công cụ **`ast_edit`** (biến đổi trực tiếp trên cây cú pháp trừu tượng) kết hợp **`lsp`** để đổi tên và cập nhật chữ ký hàm — toàn bộ các đường dẫn import, barrel files (`index.ts`) hay package Java đều được LSP cập nhật chuẩn xác 100% mà không tốn token sinh lại toàn bộ file.
3. Nếu cần chia cho 4 Subagents sửa song song 4 modules khác nhau, công cụ **`task`** sẽ tự động tách 4 Git Worktrees, kiểm tra kết quả trả về theo Schema và hợp nhất lại mà không bao giờ xảy ra xung đột ghi đè file.

3 **Kịch bản 3: Phân tích Hiệu năng & Trace Forensics bằng Loopback Eval Kernel**

Tận dụng Kernel Python/Bun gọi ngược vào Tool của Agent

Khi cần phân tích một tệp `cpu-profile.cpuprofile`, `heap-snapshot` hoặc log dữ liệu lớn (Playbook `Trace forensics`):

1. Thay vì nhồi tệp JSON nặng 50MB vào cửa sổ chat của LLM, Agent sử dụng công cụ **`eval`** của `omp` (khởi chạy persistent Python hoặc Bun worker).
2. Điểm độc đáo của `omp` là từ bên trong đoạn code Python/JS đang chạy trong `eval`, nó có thể **gọi ngược lại các công cụ `tool.read`, `tool.search`, `tool.task` của Agent qua cầu nối loopback**!
3. Kernel Python tự lọc ra 5 hàm ngốn CPU nhất từ file trace, tự gọi `tool.read` để lấy mã nguồn của 5 hàm đó, rồi chỉ trả bảng kết quả tinh gọn về cho mô hình `slow` (Opus/Grok) phân tích.

4 **Kịch bản 4: Tích lũy Trí tuệ Hệ thống qua Hindsight Memory (Retain / Recall / Reflect)**

Xây dựng Trí nhớ Dài hạn cho Codebase mà không làm phình AGENTS.md

Sau khi hoàn tất một nhiệm vụ khó (như tìm ra một quy ước ngầm của hệ thống cũ), thay vì phải viết tay vào tài liệu:

1. Agent tự động gọi công cụ **`retain`** để lưu các sự kiện thực tế (facts) và bài học tái sử dụng vào ngân hàng bộ nhớ **Hindsight** của dự án.
2. Ở các phiên làm việc ngày hôm sau, khi bạn gõ yêu cầu mới, `omp` tự động nạp mô hình tư duy thu gọn từ phiên trước và dùng **`recall`** / **`reflect`** để truy xuất đúng các bài học liên quan tới phân hệ đang mở mà không cần đọc lại toàn bộ lịch sử Git.

4.3. Sơ đồ Tương tác: Luồng Điều phối Yêu cầu trên Kiến trúc `omp` + `pstack`

Sơ đồ tương tác dưới đây mô phỏng chi tiết từng bước một yêu cầu kỹ thuật đi qua bộ định tuyến `/poteto-mode`, hệ thống phân vai 9 mô hình (`modelRoles`), bộ chặn dòng thời gian `TTSR`, cụm công cụ IDE nội tại (`LSP`/`AST`/`DAP`) và hạm đội `task` Worktree Subagents trong `omp`:
